Lab 5 : Unsupervised Learning - Searching for a new galaxy

The goal of this lab is to:

- Work on several examples of unsupervised learning.
- Use K-means clustering to perform classification.
- Use Gaussian mixture modeling to for classification and density estimation.
- Determine the optimal number of clusters to use for a problem.
- Identify the remants of a dwarf galaxy that merged with the Milky Way using velocity and metal abundance measurements of stars from the Gaia survey.

Created by: Tuan Do & Bernie Boscoe Last update: Tuan Do

In [66]:
```
# put your imports here
import numpy as np
import pylab as plt
import pandas as pd
from sklearn.datasets import make_blobs
import seaborn as sns
```

Part 1 - K-Means Clustering

K-means clustering is a simple clustering algorithm, which contains just a few steps:

1. Guess some cluster centers
2. Repeat the following until converged
    - Expectation step – assign points to the nearest cluster center
    - Maximization step – set the cluster centers to the mean.

Convergence is defined as when the centroid of the cluster does not change after Step 2.

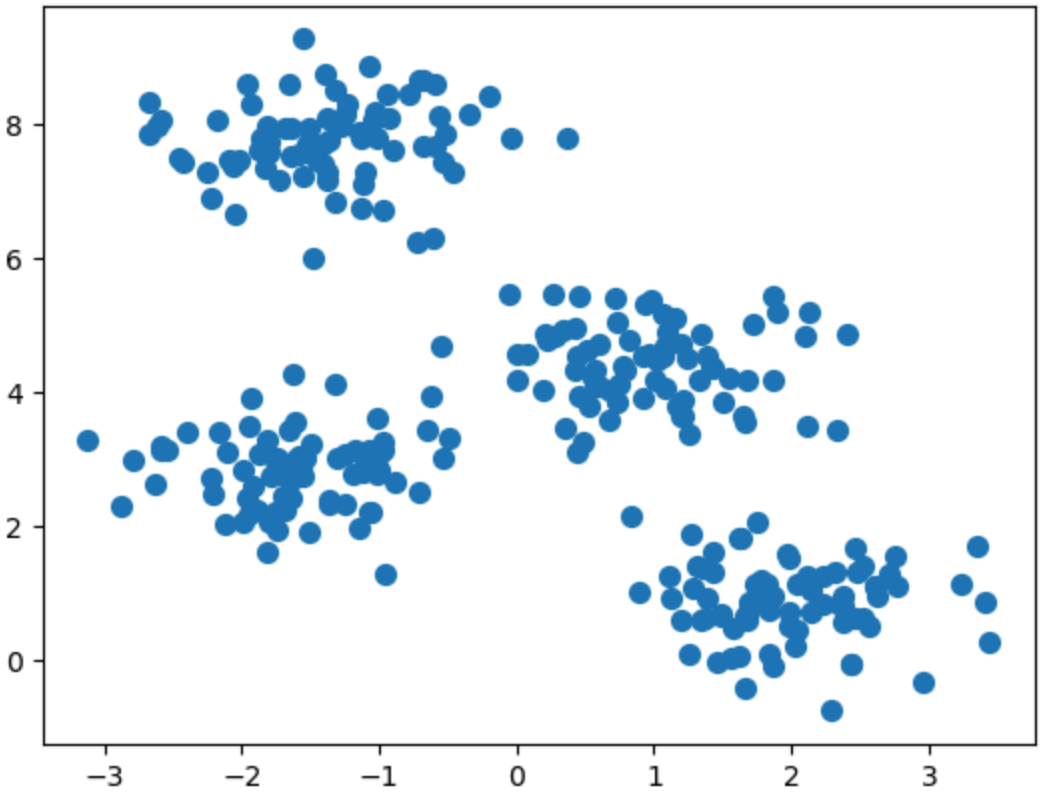
In this part of the lab, you will generate random clusters and practice using K-means clustering to learn about the model and its performance.

Part 1.1 - Generating random clusters

We will generate first a random set of clusters with 2 features.

In [2]:
```
#from sklearn.datasets.samples_generator import make_blobs
X, y_true = make_blobs(n_samples=300, centers=4,
```

```
                           cluster_std=0.60, random_state=0)
plt.scatter(X[:, 0], X[:, 1], s=50);
```



Part 1.2 - Building and training a K-means classifier

Question 1

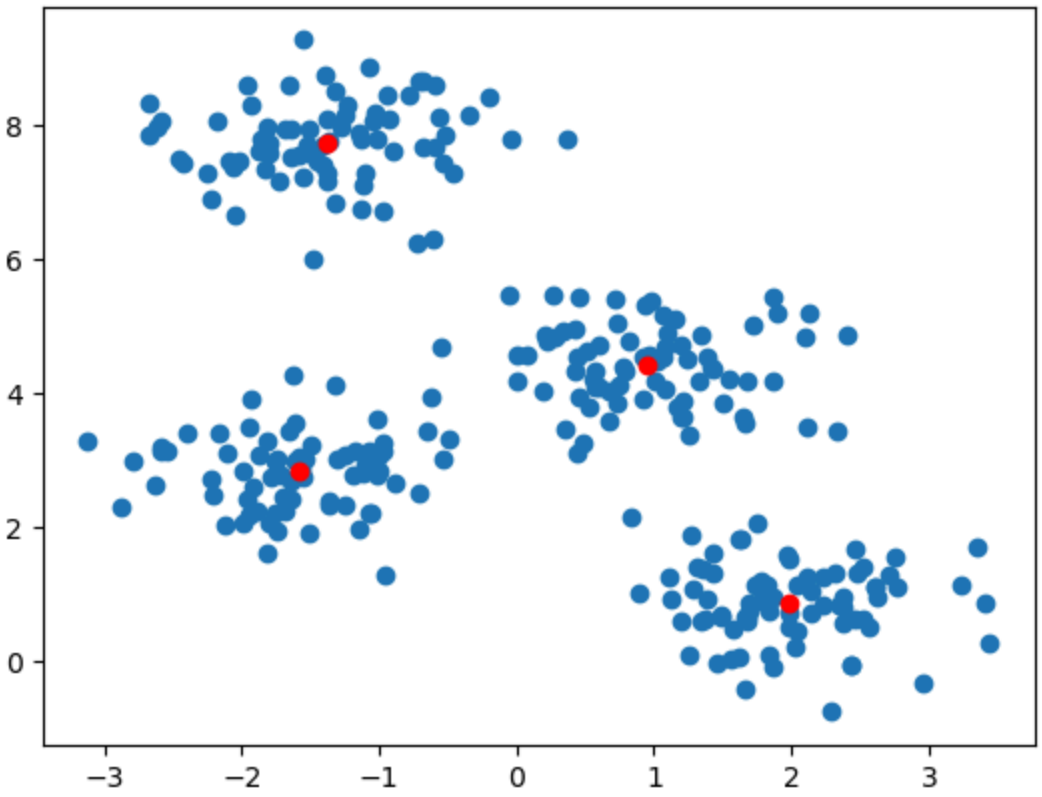
(5 pts)

Now, read the K-means documentation in sklearn (https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html) and create a K-means classifier and train it.

In [3]:
```
# put code here
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4, random_state=0, n_init="auto").fit(X)
kmeans.labels_
kmeans.predict([[0, 0], [12, 3]])
means=kmeans.cluster_centers_
```

Visualize your fit - plot the different detected clusters

In [6]:
```
# enter code here
plt.scatter(X[:,0],X[:,1])
plt.scatter(means[:,0],means[:,1],color='red')
plt.show()
```



Four different clusters predicted !!

Decision Boundaries below is the code to help you visualize the decision boundaries of your model. Run it and make a plot for your clusters.

In [7]:
```
#  Code to plot decision boundaries for K-means

def plot_data(X):
    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2)

def plot_centroids(centroids, weights=None, circle_color='w',
cross_color='k'):
    if weights is not None:
        centroids = centroids[weights > weights.max() / 10]
    plt.scatter(centroids[:, 0], centroids[:, 1],
                marker='o', s=35, linewidths=8,
                color=circle_color, zorder=10, alpha=0.9)
    plt.scatter(centroids[:, 0], centroids[:, 1],
                marker='x', s=2, linewidths=12,
                color=cross_color, zorder=11, alpha=1)

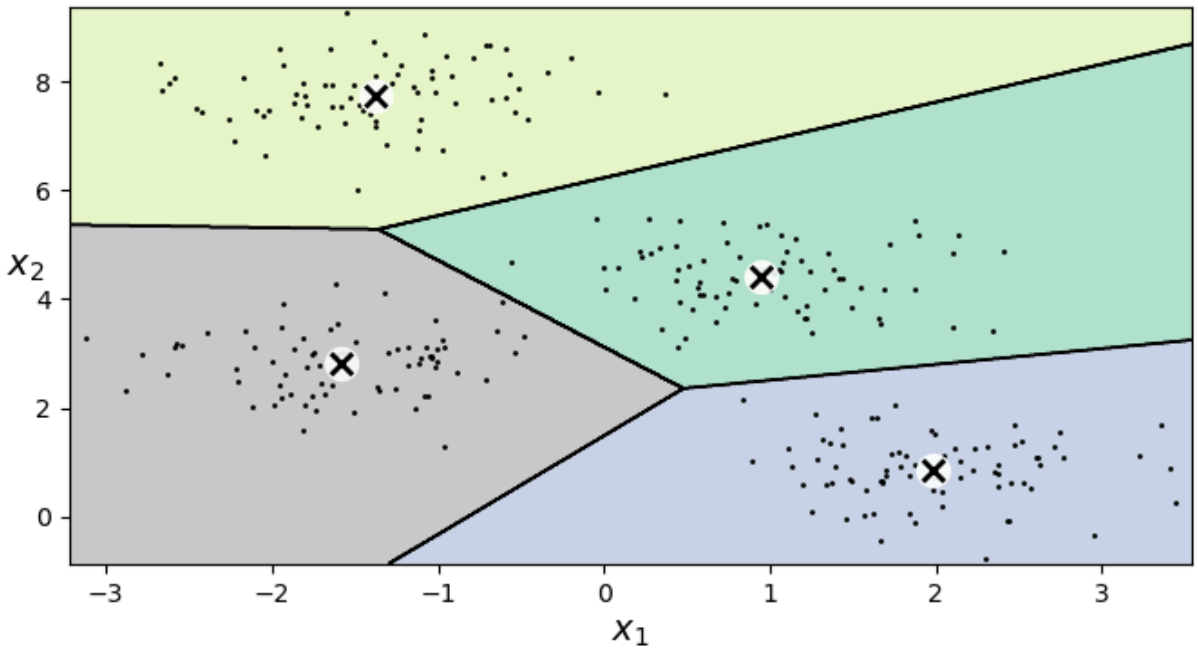
def plot_decision_boundaries(clusterer, X, resolution=1000,
show_centroids=True,
                             show_xlabels=True, show_ylabels=True):
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution),
                         np.linspace(mins[1], maxs[1], resolution))
    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
```

```
    Z = Z.reshape(xx.shape)

    plt.contourf(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                 cmap="Pastel2")
    plt.contour(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                linewidths=1, colors='k')
    plot_data(X)
    if show_centroids:
        plot_centroids(clusterer.cluster_centers_)

    if show_xlabels:
        plt.xlabel("$x_1$", fontsize=14)
    else:
        plt.tick_params(labelbottom=False)
    if show_ylabels:
        plt.ylabel("$x_2$", fontsize=14, rotation=0)
    else:
        plt.tick_params(labelleft=False)
```

In [8]:
```
# visualize the decision boundaries
plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X)
```



Part 1.2 - fitting a different number of cluster

The number of clusters is a hyperparameter for the K-means clustering algorithm.

What will happen if we fit 3 clusters instead of 4 clusters?

Question 2

(2 pts)

Before writing code, **predict where you think the centroid of the 3 clusters will be and discuss why.** A rough estimate of the centroid is fine.

put answer here

I think the the number of clusters should be 3. Just eyeballing the data it seems that the two clusters in the center of the visualized plot are closer together than the other two clusters sitting in the bottom right and top left. Accordingly, if I were to predict clustering into three groups, I would surmise that the algorithms clusters the the middle blob as one cluster ! I

Question 3

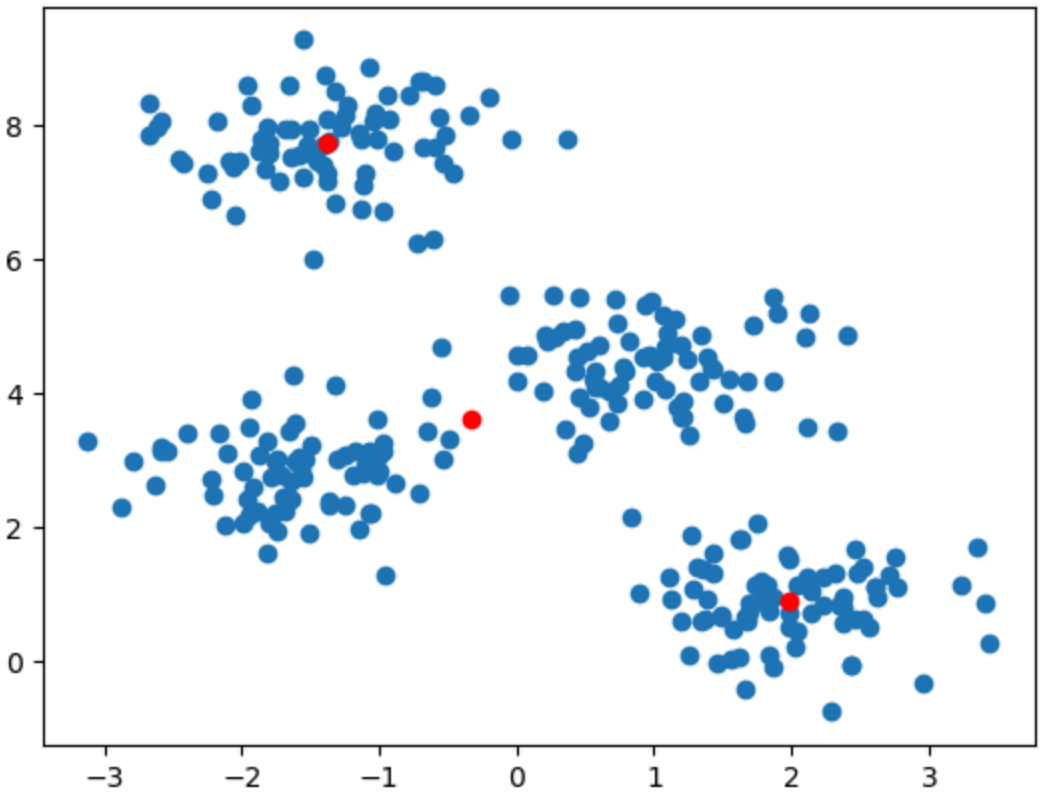
(5 pts)

Below, write the code to fit for 3 clusters instead of 4 to the same dataset as above using the K-means algorithm.

In [11]:
```
# code here
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=0, n_init="auto").fit(X)
kmeans.labels_
kmeans.predict([[0, 0], [12, 3]])
means=kmeans.cluster_centers_
```

Now, visualize the fit with only 3 clusters

In [12]:
```
# put code here
plt.scatter(X[:,0],X[:,1])
plt.scatter(means[:,0],means[:,1],color='red')
plt.show()
```



Our naive theoretical intuition of the how three clusters would form is correct !

Part 1.3 - Determining the Optimal Number of Clusters for K-Means

It can be difficult to determine the correct number of clusters because the definition of a cluster can be pretty vauge. There are a few guidelines we methods that we can use though to help us. We will follow the two methods used in Hands-On Machine Learning 2nd Edition:

1. Look for the 'elbow' in the plot of the number of clusters vs. interia.
2. Look at the silhouette score as a function of the number of clusters.

Part 1.3a - Plot the intertia as a function of number of clusters

Question 4

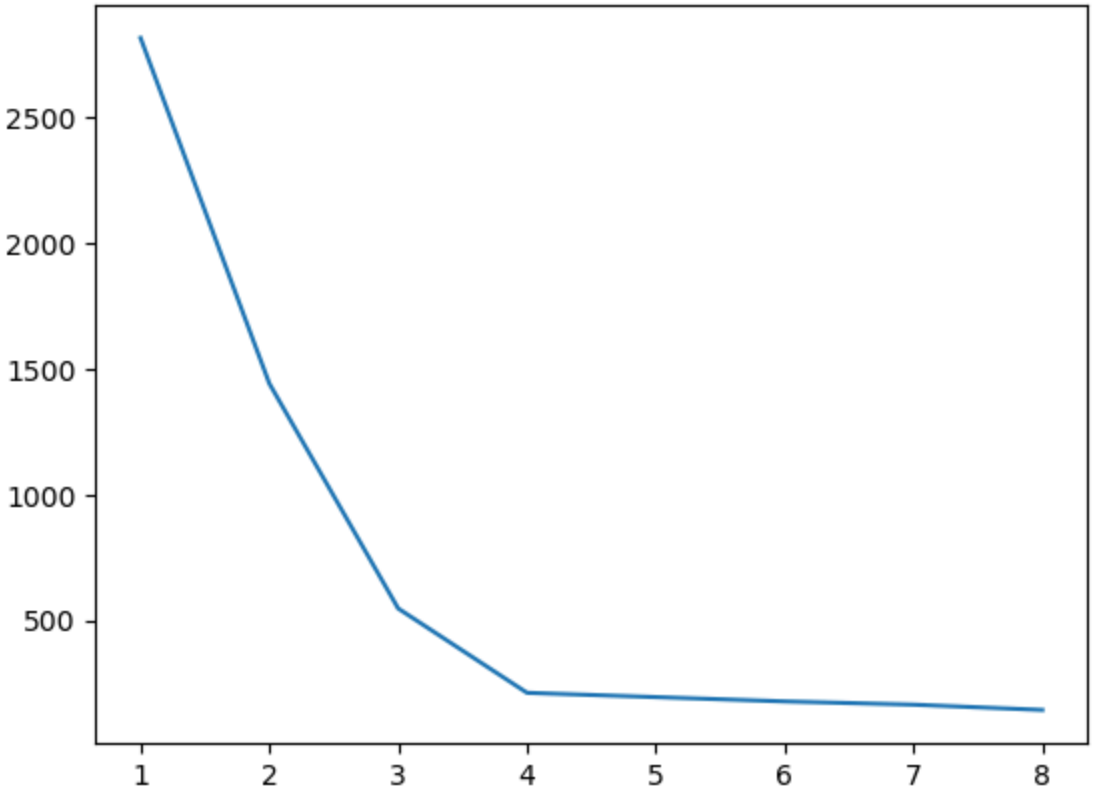
(2 pts)

Once the model is trained the kmeans object from sci-kit learn has an `inertia_` attribute that measures the inertia of the fit (see https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html).

Make a plot of the interia as a function of number of clusters (from 1 to 8 clusters)

In [13]:
```
# your code here
inertias=[KMeans(n_clusters=i,random_state=0,n_init='auto').fit(X).inertia_
for i in range(1,9)]
```

In [14]:
```
plt.plot(np.arange(1,9),inertias)
plt.show()
```



In the plot above, there should be a point on the plot where it becomes flat. This point is called the elbow and is a good candidate for the optimal number of clusters in your model.

Question 5

(2 pts)

Where is the elbow for your dataset? Explain qualitatively why the curve flatens out (remember the definition of inertia).

In [ ]:
```
# you answer here
```

The elbow is clear to see - 4 optimal clusters !!

Part 1.3b - Using the silhouette coefficient

Another method to find the optimal number of clusters is to use the silhoueet coefficient.

See the documentation here: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html

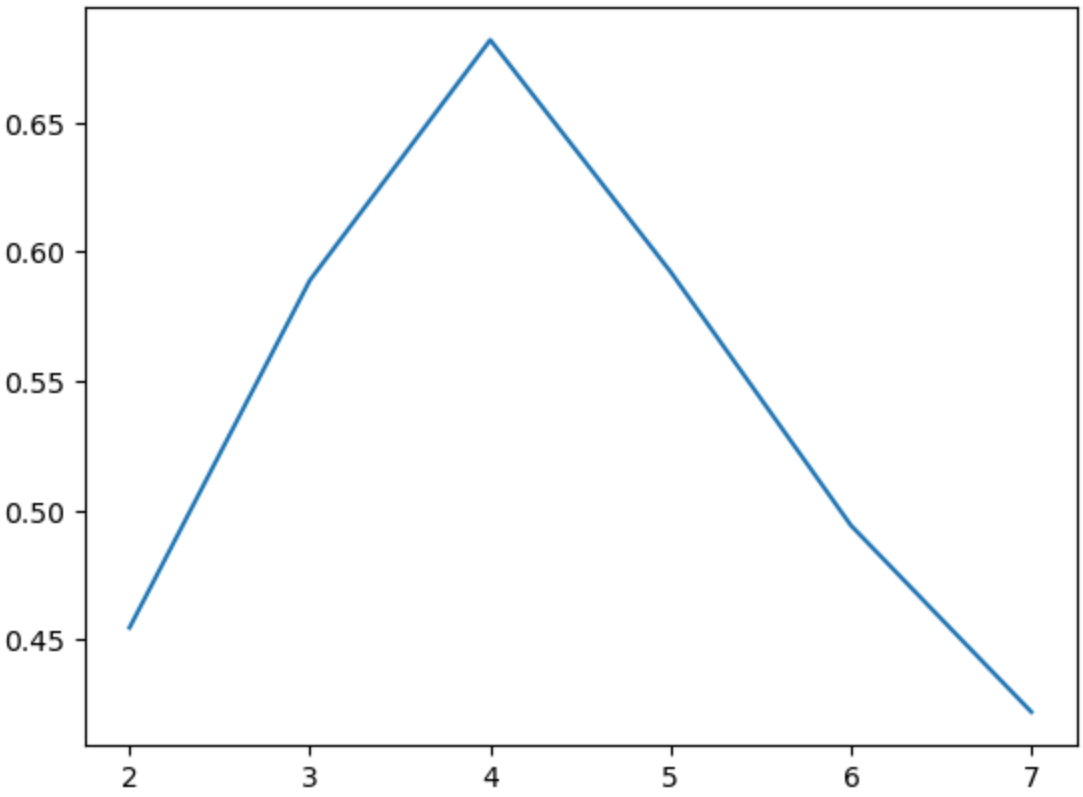
Question 6

(2 pts)

Plot below the silhouette coefficient for the your dataset as a function of the number of clusters (from 1 to 8)

In [17]:
```
# your code here
from sklearn.metrics import silhouette_score
silhouettes=[silhouette_score(X,KMeans(n_clusters=i, random_state=42,
n_init='auto').fit_predict(X)) for i in range(2,8)]
```

In [18]:
```
# your answer here
plt.plot(np.arange(2,8),silhouettes)
plt.show()
```



In the plot above, what is the best sihouette score? Explain qualitatively why the sihouette score peaks where it does.

It is clear that the peak is at 4 clusters !

When the number of clusters is too small, say $2, 3$, multiple blobs get lumped into the same cluster, dragging the score down ! On the other hand, when the number of clusters is too large, say $5, 6$, the algorithms starts splitting real blobs into smaller pieces to create artificial clusters. This shrinks the gap between neighbouring clusters and the score drops again ! The peak at 4 clusters seems to be a sweet splot where the cluster assignments best reflect the true underlying structure of the dats !

Part 2 - Gaussian Mixture Modeling

Gaussian Mixture Modeling is a lot more flexible than K-means. It can be used for cluster, but also for modeling the probability distribution of the data points. Gaussian mixture modeling can be thought of as fitting N multi-densional Gaussian with M dimensions where M is the nubmer of features in the data.

For this part of the lab, we will explore how to use sci-kit learn's Gaussian model, how to visualize the fitted model, make predictions from the model, and determine the optimial number of clusters.

Part 2.1 - Generating data

Create the dataset by running the code below.

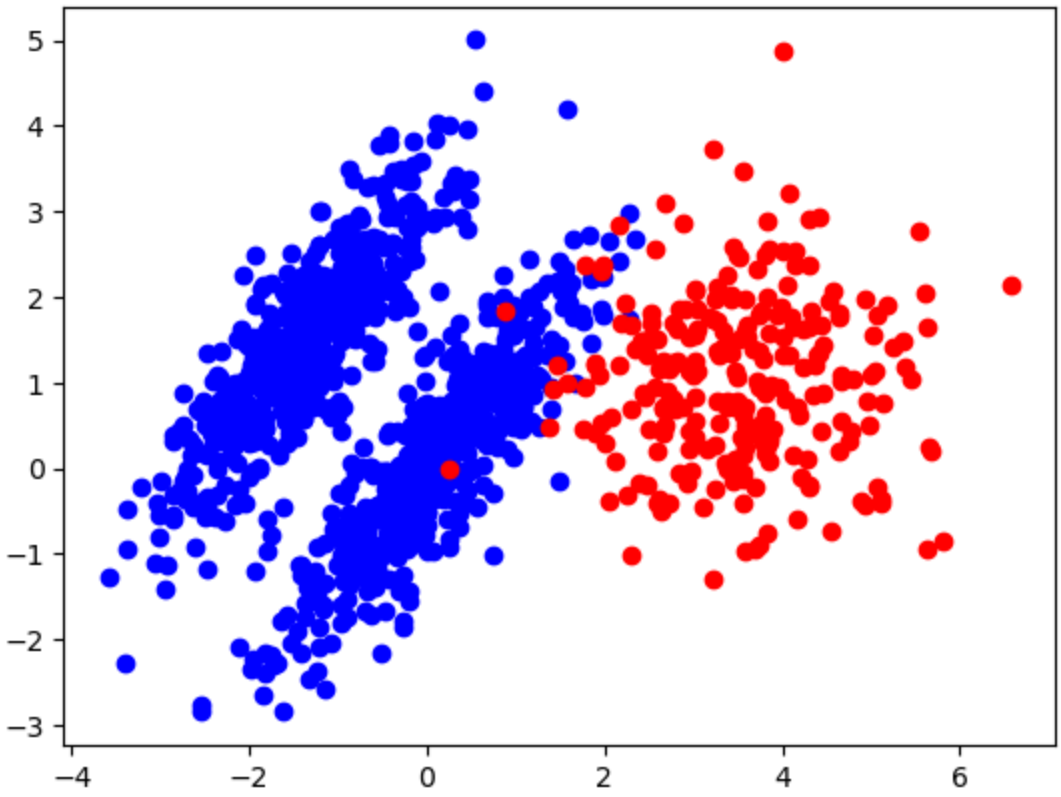
In [19]:
```
# create the dataset
X1, y1 = make_blobs(n_samples=1000, centers=((4, -4), (0, 0)),
random_state=42)
X1 = X1.dot(np.array([[0.374, 0.95], [0.732, 0.598]]))
X2, y2 = make_blobs(n_samples=250, centers=1, random_state=42)
X2 = X2 + [6, -8]
X = np.r_[X1, X2]
y = np.r_[y1, y2]
```

Question 7

(2 pts)

Visualize the dataset - plot the `X` arrays of features. Describe where you would place the cluster center by eye.

In [20]:
```
# enter code here
plt.scatter(X1[:,0],X1[:,1],color='blue')
plt.scatter(X2[:,0],X2[:,1],color='red')
plt.show()
```



By eye, I'd cluster the data into three clusters - two blue blobs an one red blob !!

Part 2.2 - Fitting a Gaussian Mixture model

Read the documentation for sci-kit learn's implementation of the Gaussian mixture model:

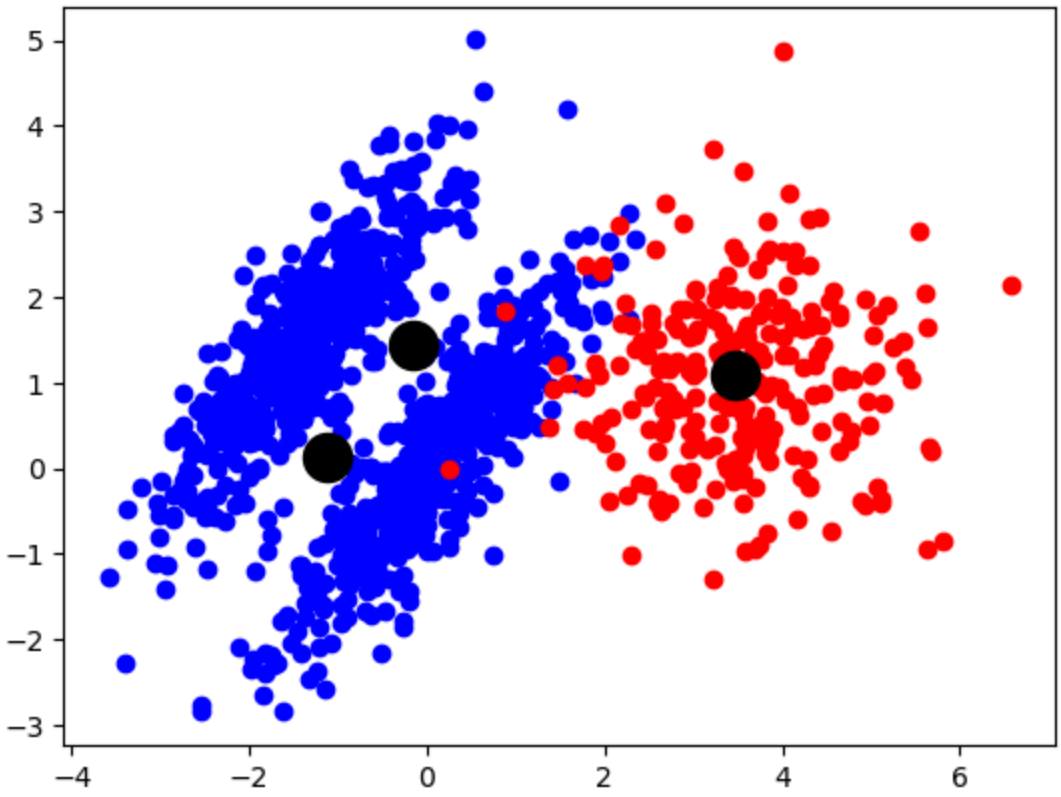
https://scikit-learn.org/stable/modules/generated/sklearn.mixture.GaussianMixture.html

Question 8

(3 pts)

Use the sci-kit learn Gaussian mixture model to fit the data above.

In [23]:
```
# your code here
from sklearn.mixture import GaussianMixture
gm = GaussianMixture(n_components=3, random_state=0).fit(X)
gmmeans=gm.means_
plt.scatter(X1[:,0],X1[:,1],color='blue')
plt.scatter(X2[:,0],X2[:,1],color='red')
plt.scatter(gmmeans[:,0],gmmeans[:,1],color='black',s=300)
plt.show()
```



The black circles represnt the center of the three Gaussians !

Question 9

(2 pts)

Examine the following attributes of your model and explain what they mean

- `.weights_`
- `.means_`
- `.covariances_`
- `.converged_`
- `.n_iter_`

In [24]:
```
# your answer here
gm.weights_
```

Out[24]:
```
array([0.3803392 , 0.20131947, 0.41834133])
```

Calcualtes the proportion of data points belonging to each Gaussian component and must sum to 1.

In [25]:
```
gm.means_
```

Out[25]:
```
array([[-0.15096303,  1.44181791],
       [ 3.46177834,  1.09542904],
       [-1.12344797,  0.12784253]])
```

The center of each Gaussian component !

In [26]:
```
gm.covariances_
```

Out[26]:
```
array([[[ 1.08310295, -0.10906453],
        [-0.10906453,  1.13580806]],

       [[ 1.07309918, -0.08248788],
        [-0.08248788,  0.97414027]],

       [[ 0.96090769, -0.03335572],
        [-0.03335572,  1.24997492]]])
```

The shape and orientation of each Gaussian component : how spread out each cluster is !

In [27]:
```
gm.converged_
```

Out[27]:
```
True
```

Tests whether the algorithm converged successfully before hitting the iteratio limit - True in this case !!!

In [28]:
```
gm.n_iter_
```

Out[28]:
```
7
```

Calculates the number of iterations it took to reach convergence !

Visualize the model - run the following cells to visualize the predictions from the Gaussian mixture model

In [29]:
```
from matplotlib.colors import LogNorm

def plot_gaussian_mixture(clusterer, X, resolution=1000,
show_ylabels=True):
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution),
                         np.linspace(mins[1], maxs[1], resolution))
    Z = -clusterer.score_samples(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z,
                 norm=LogNorm(vmin=1.0, vmax=30.0),
                 levels=np.logspace(0, 2, 12))
    plt.contour(xx, yy, Z,
                norm=LogNorm(vmin=1.0, vmax=30.0),
```

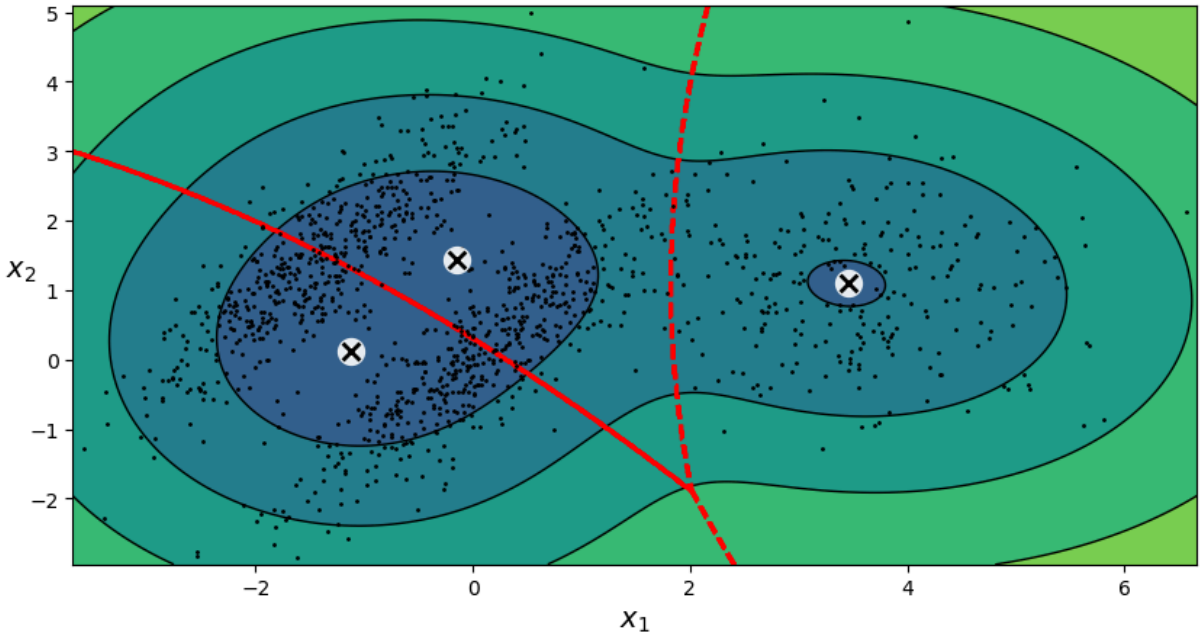
```
                 levels=np.logspace(0, 2, 12),
                 linewidths=1, colors='k')

    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contour(xx, yy, Z,
                linewidths=2, colors='r', linestyles='dashed')

    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2)
    plot_centroids(clusterer.means_, clusterer.weights_)

    plt.xlabel("$x_1$", fontsize=14)
    if show_ylabels:
        plt.ylabel("$x_2$", fontsize=14, rotation=0)
    else:
        plt.tick_params(labelleft=False)
```

In [31]:
```
plt.figure(figsize=(10, 5))
plot_gaussian_mixture(gm, X)
```



Part 2.3 - Prediction and Generating new instances using the model

Since Gaussian mixture models are probabilistic, we can use it to determine the probability of belonging to a given cluster for any instance of the dataset. We can also generate new instances of data given a trained model.

Question 10

(4 pts)

Predict the class and probability of each instance of the training data. Compare the predictions with the number of instances you expected to be in each cluster

Hint: use the `.predict()` and `.predict_proba()` methods

In [32]:
```
# your code here
# predict the class and probability
y_pred = gm.predict(X)
y_proba=gm.predict_proba(X)
```

In [33]:
```
# count predicted instances per cluster
unique,counts=np.unique(y_pred,return_counts=True)
print("Predicted counts per cluster:",dict(zip(unique,counts)))
```

```
Predicted counts per cluster: {0: 469, 1: 250, 2: 531}
```

In [34]:
```
# compare with expected counts
print("Expected count for X1:",len(X1))
print("Expected count for X2:",len(X2))
print("Total counts:",len(X))
```

```
Expected count for X1: 1000
Expected count for X2: 250
Total counts: 1250
```

In [35]:
```
# we show a sample for probabiltis
print("\nSample predicted probabilities, say first five rows:")
print(y_proba[:5])
```

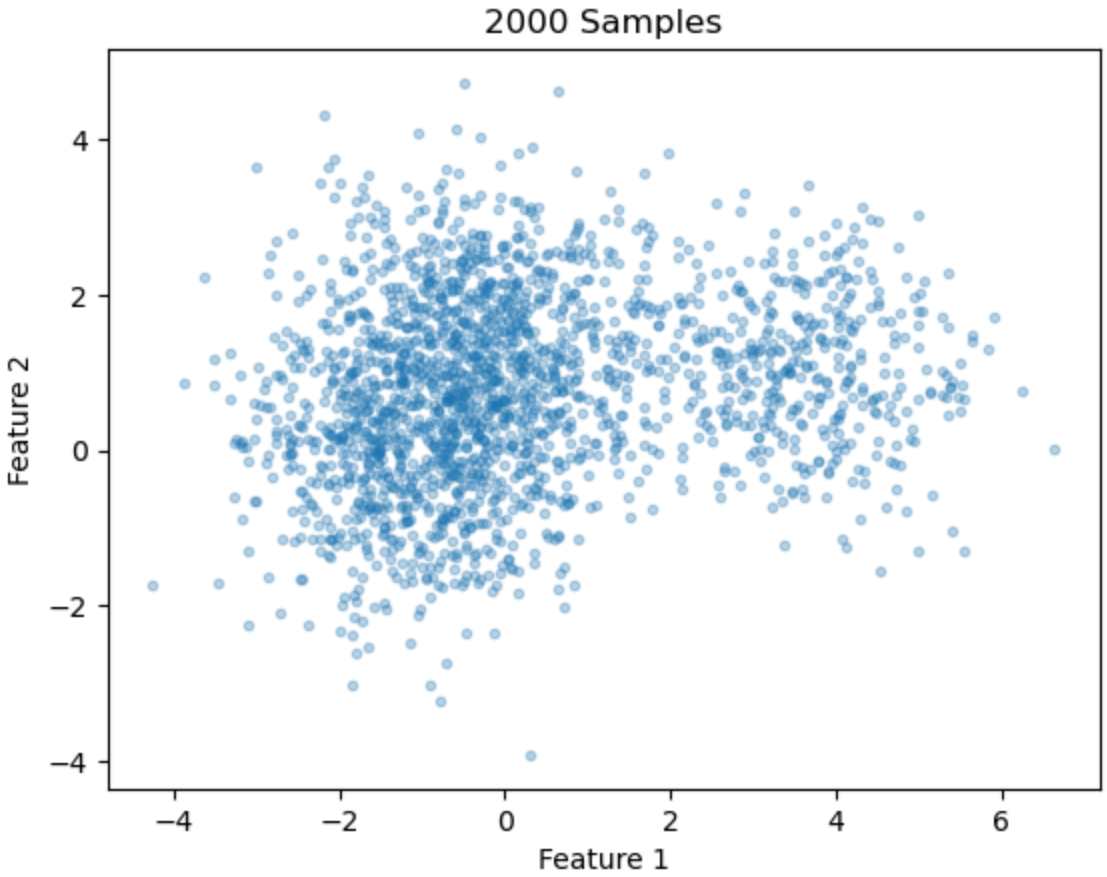
```
Sample predicted probabilities, say first five rows:
[[8.61617225e-01 2.92129474e-02 1.09169828e-01]
 [8.04944147e-01 4.61318570e-03 1.90442668e-01]
 [8.58939507e-02 3.06199002e-07 9.14105743e-01]
 [7.80217945e-01 6.99738802e-03 2.12784667e-01]
 [3.39832214e-02 6.66293280e-09 9.66016772e-01]]
```

Generate 2000 new samples of the dataset and plot them

Hint: use the `.sample` method in the trained model to generate instances.

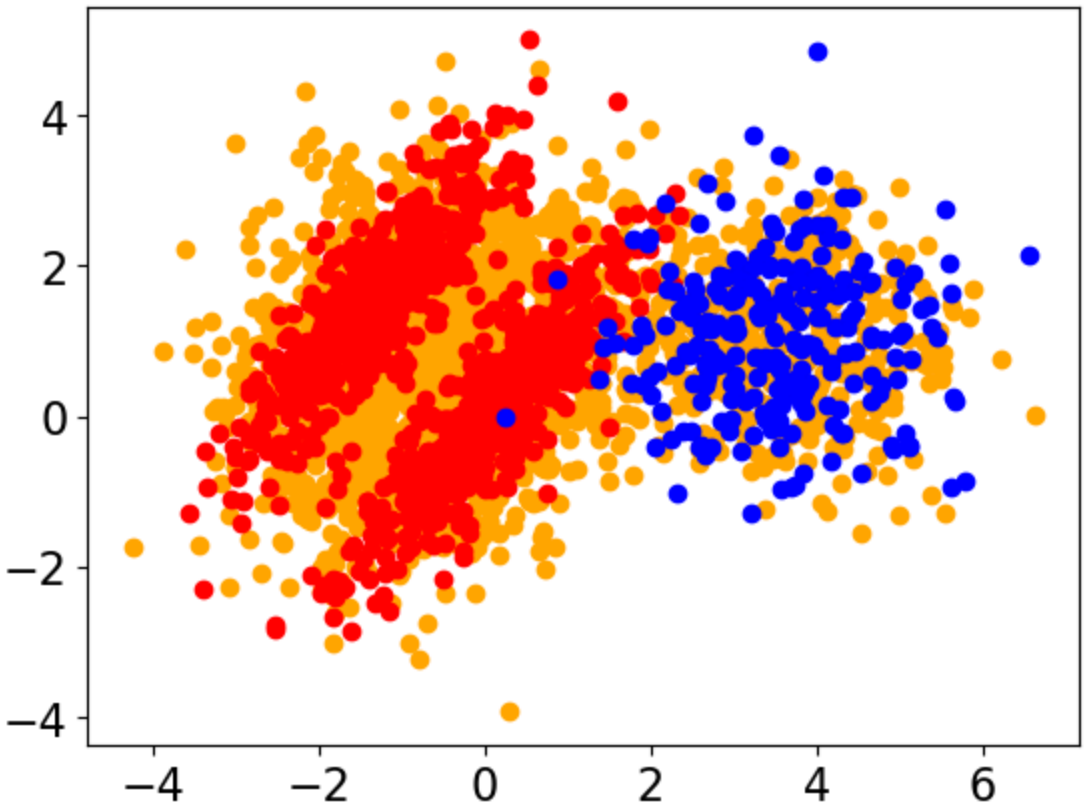
In [44]:
```
# your code here
gmsamples = gm.sample(2000)
samples = gmsamples[0]

plt.scatter(samples[:,0], samples[:,1], alpha=0.3, s=10)
plt.title('2000 Samples')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



Samples only, with transparency to reveal density variation across the sampled points !!

In [41]:
```
plt.scatter(samples[:,0],samples[:,1],color ='orange')
plt.scatter(X1[:,0],X1[:,1],color='red')
plt.scatter(X2[:,0],X2[:,1],color='blue')
plt.show()
```



Samples in orange overlaid with original data (red and blue), showing how well the model learned the underlying distribution !!!

Part 2.4 Determining the Optimal Number of Clusters

Gaussian mixtures are a probabilistic model, so we can use our Bayesian inference framework for things like model selection. In this case, we want to compare models with different number of clusters and select the one that best explain the data with the fewest parameters.

There are two Bayesian model selection criteria that are built into sci-kit learn:

- Bayesian Information Criterion
- Akaike Information Criterion

Both of these criteria are described in Chapter 9 of Hands-On ML 2nd Edition. For both of these information criteria, you want the model that has the lowest value.

NOTE: you can also use these criteria to select between different kinds of Gaussian models. For example, the sci-kit learn Gaussian mixture model has different choices for the type of covariance to fit using the `covariance_type` keyword.

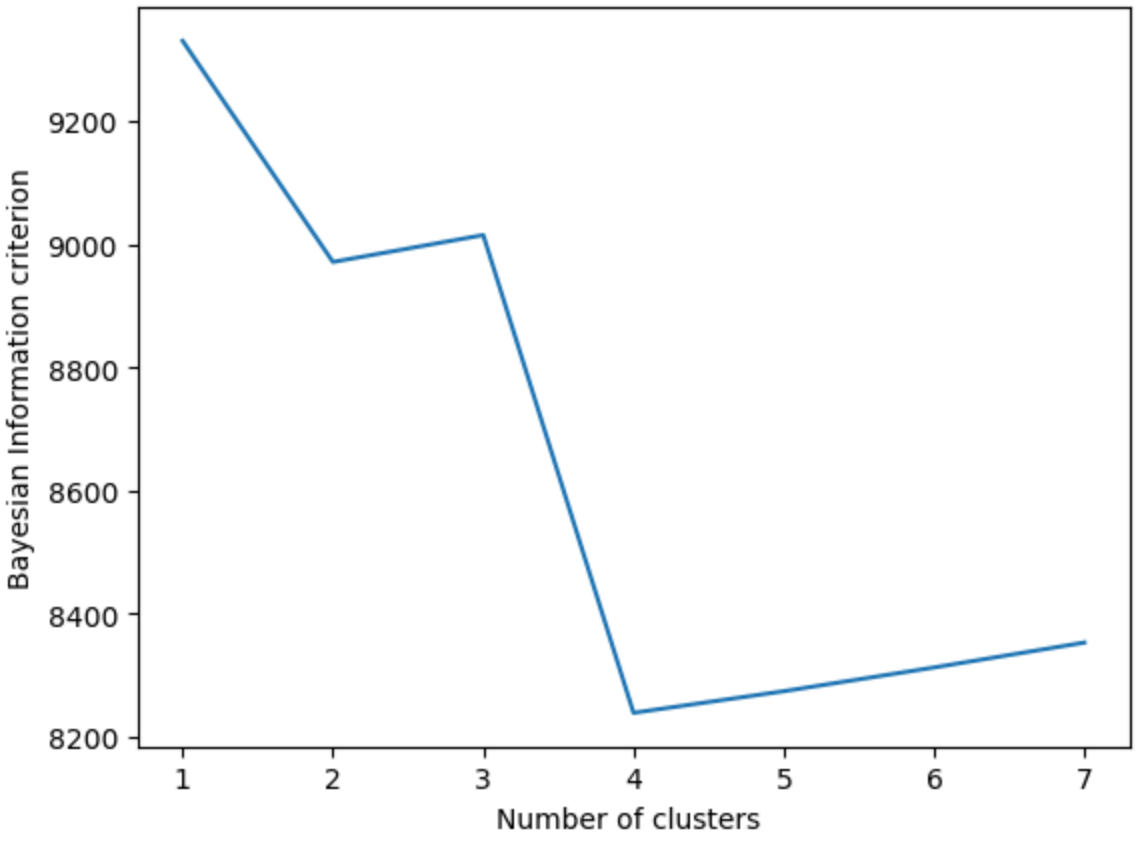
Question 11

(4 pts)

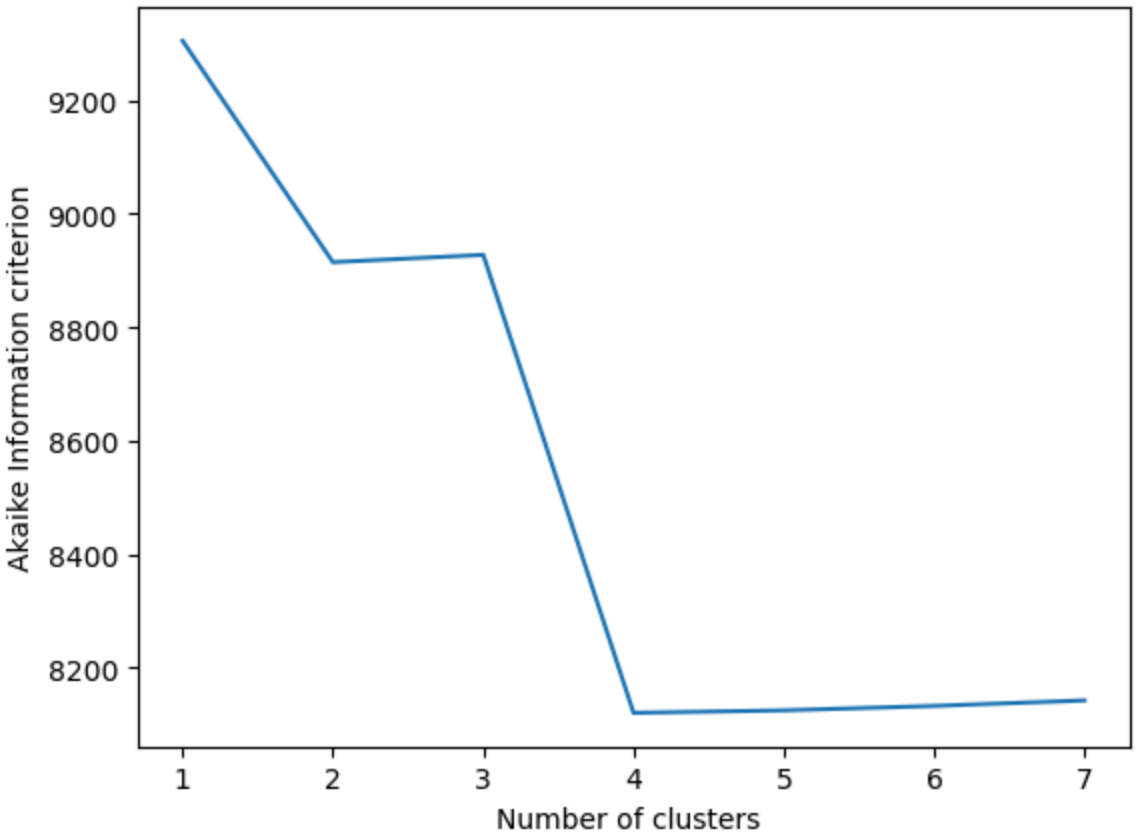
Compute the BIC and the AIC for a Gaussian mixture model with between 1 to 8 clusters. What is the optimal number of clusters?

In [46]:
```
# your code here
bics=[GaussianMixture(n_components=i, random_state=0).fit(X).bic(X) for i
in range(1,8)]
aics=[GaussianMixture(n_components=i, random_state=0).fit(X).aic(X) for i
in range(1,8)]
```

In [51]:
```
plt.plot(np.arange(1,8),bics)
plt.ylabel("Bayesian Information criterion")
plt.xlabel("Number of clusters ")
plt.show()
```



In [52]:
```
plt.plot(np.arange(1,8),aics)
plt.ylabel("Akaike Information criterion")
plt.xlabel("Number of clusters ")
plt.show()
```



Optimal number of clusters - 4 !! This is different from what we predicted by eye (3).

The BIC and AIC minimum at 4 is the mathematically optimal answer given our data and it's consistent with how the blobs were originally generated ! What likely looks like 3 clsuters visually is because two of the blobs may be close together or overlapping, making them appear as one to the eye but still statistically distinct enough for BIC to separate them !

Part 2.5 - Fitting for both the cluster parameters and the number of clusters at the same time

Since we are using a Bayesian framework, we can also fit for the number of clusters directly in our inference for the cluster parameters. In fact, this is generally how you would want to try in real-world data as it tends to be more robust to fit everything together.

Luckily sci-kit learn has this capability for Gaussian mixture modeling through the `BayesianGaussianMixture` model. Read the documentation at: https://scikit-learn.org/stable/modules/generated/sklearn.mixture.BayesianGaussianMixture.html

Question 12

(4 pts)

Fit the dataset above using the `BayesianGaussianMixture` model

Note that to use `BayesianGaussianMixture`, you will need to give it an initial guess for the number of clusters. Make this initial guess large (like 10).

In [57]:
```
# your code here
from sklearn.mixture import BayesianGaussianMixture
initial_guess=10
bgm = BayesianGaussianMixture(n_components=initial_guess,
random_state=42).fit(X)
bgm.means_
y_pred=bgm.predict(X)
```

```
/Users/ameygaikwad/anaconda3/lib/python3.11/site-packages/sklearn/mixture/_base.py:269: ConvergenceWarning: Best performing initialization did not converge. Try different init parameters, or increase max_iter, tol, or check for degenerate data.
  warnings.warn(
```

Examine the `.weights_` attribute after fitting. What does it mean?

In [58]:
```
# your code here
bgm.weights_
```

Out[58]:
```
array([3.95127351e-01, 2.05029810e-01, 8.51923159e-04, 3.98832938e-01,
       1.44798853e-04, 1.19811223e-05, 1.08919211e-06, 9.90174642e-08,
       9.00158766e-09, 8.18326151e-10])
```

The weights show that only 4 components have significant values (close to the true number of clusters), while the remaining components have weights near zero. The Bayesian model automatically turned off the unnecessary components it did not need, despite being initialized with 10.

Part 3 - Discovering a new dwarf galaxy embedded in the Milky Way

The Gaia spacecraft and the all sky survey that it is producing is changing many aspects of astronomy. The Gaia catalog now contains over 2 billion sources with position and velocity measurements.

One of the big discoveries of Gaia is the detection of the remnant of a dwarf galaxy embedded in the Milky Way that was previously hidden from view because the stars are mixed with the Milky Way stars. This dwarf galaxy is called Gaia-Enceladus (also known as the Gaia Sausage). More details can be found in the discovery paper from Helmi et al. (2018): https://arxiv.org/abs/1806.06038

In this part of the lab, you will use a subset of the Gaia dataset to see if you can detect Gaia-Enceladus using unsuperivsed clustering methods. Unlike the other clustering excercises you did in other parts of the lab, this time, you'll use more than two features.

3.1 Big Picture

The science question we would like to answer is: "How did the components of the Milky Way form?" To answer this question, we can test models of the formation of the Milky Way. One model is that the Milky Way halo is made up of the mergers of smaller galaxies. Evidence of past mergers would support this model.

3.2 Get the data

In this lab's directory, there's a file called `gaia_apogee_match_noflag.csv`. This file contains data from the Gaia survey with the kinematic information on the stars. It contains data from the APOGEE survey, which is a spectroscopic survey from the Sloan Digital Sky Survey. APOGEE provides measurements of the elemental abundances for these stars.

The columns are:

- source_id - ID number of the star in Gaia
- vx - velocity of the star in the direction of the Galactic center
- vy - tangential velocity of the star in the Milky Way disk of the Milky Way
- vz - perpendicular velocity to the Milky Way disk
- En - total energy of the star
- Lz - angular moment of the stars
- apogee - the star's name in the APOGEE survey
- feh - [Fe/H] the logarithm of the iron abundance of the star as a fraction of hydrogen, scaled by the sun. ([Fe/H] = 0 for the Sun)
- alpha - [alpha/Fe] the logarithm of the ratio of alpha-elements to iron in (scaled by the sun), [alpha/Fe]= 0 for the Sun.

Question 13

(2 pts)

Load the data into a pandas data frame and examine it

In [59]:
```
# code here
# increase font size
plt.rcParams.update({'font.size': 16})
data = pd.read_csv('gaia_apogee_match_noflag.csv')
```

In [60]:
```
data.head()
```

Out[60]:

	source_id_1	vx	vy	vz	En	
0	61047157615088256	-25.761714	222.054260	29.777083	-155526.229018	2298.72
1	65723380567903104	-1.410748	222.424836	27.887632	-152480.050683	2477.46
2	66570932234426112	-52.639903	213.206085	10.694683	-159300.952613	2120.13
3	67139104867760384	23.515292	232.149817	34.817371	-150036.436055	2578.78
4	73621275666889344	-122.512619	-15.886698	7.277072	-176382.521936	-119.02

3.3 Explore the data

Question 14

(5 pts)

Check the shape, and head, plot, visualize, and have a look at the data with whatever ways you think will help your task. Comment throughout on observations. Are you there potential clustering that you see in some of these features?

In [62]:
```
# code here
# shape
print("Shape:",data.shape)
print("\nColumn names:",data.columns.tolist())
```

```
Shape: (526, 9)

Column names: ['source_id_1', 'vx', 'vy', 'vz', 'En', 'Lz', 'apogee', 'feh',
'alpha']
```

In [64]:
```
# basic stats
data.describe()
```

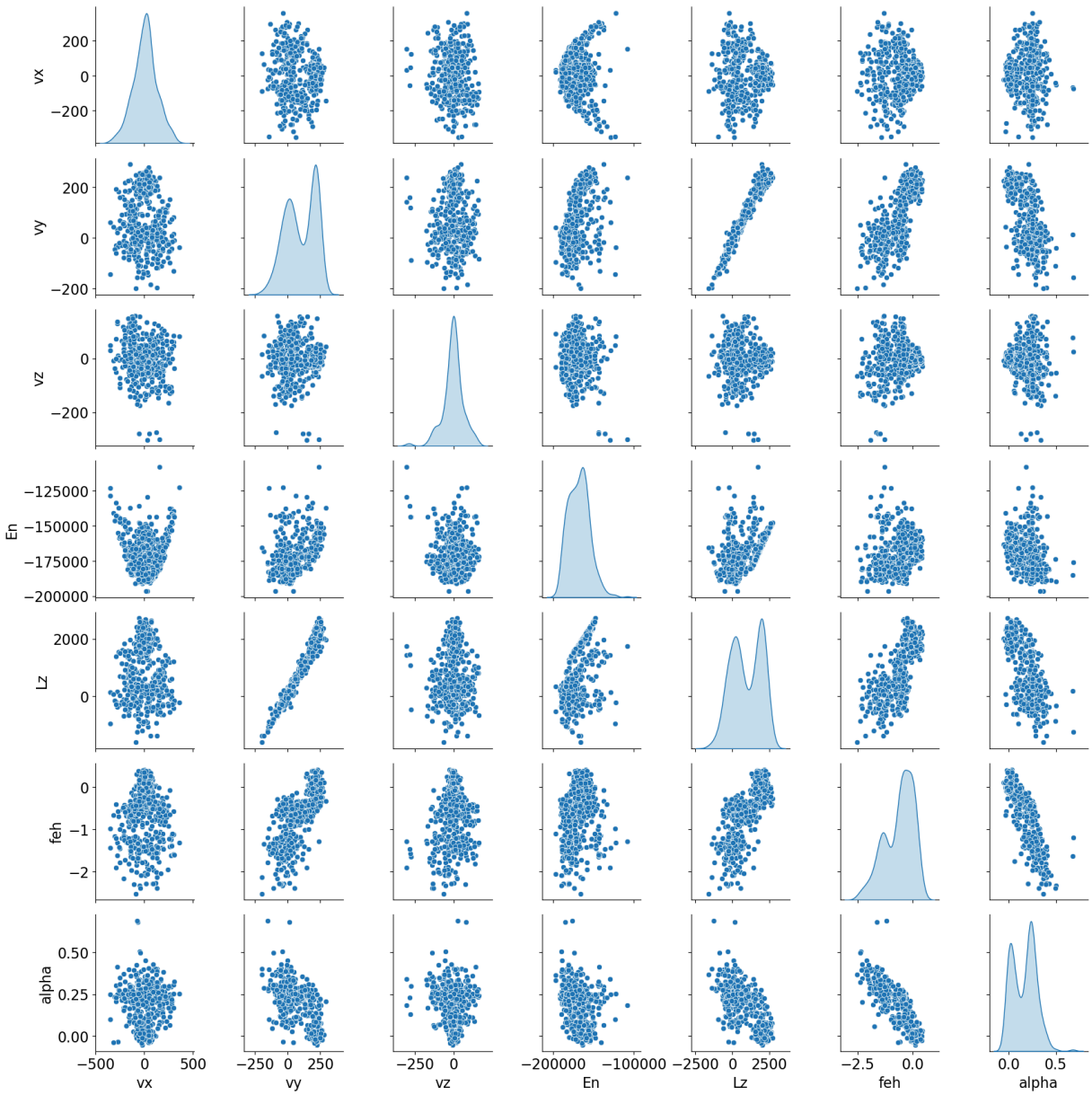
Out[64]:

	source_id_1	vx	vy	vz	En	
count	5.260000e+02	526.000000	526.000000	526.000000	526.000000	526.0(
mean	2.102764e+18	7.275274	96.949364	-5.325371	-167971.268549	949.9
std	1.309843e+18	121.910853	115.644161	64.978286	13133.246109	1010.7
min	6.104716e+16	-352.357185	-199.859889	-302.312380	-196516.693412	-1612.7
25%	1.269537e+18	-65.534483	2.101660	-30.951782	-178254.104527	101.9
50%	1.597441e+18	14.644768	93.587644	-2.765396	-167769.209163	887.8
75%	3.373111e+18	73.839982	209.344272	24.721122	-159373.840061	1938.9
max	6.917218e+18	359.676197	290.882264	159.071067	-107879.448995	2720.8

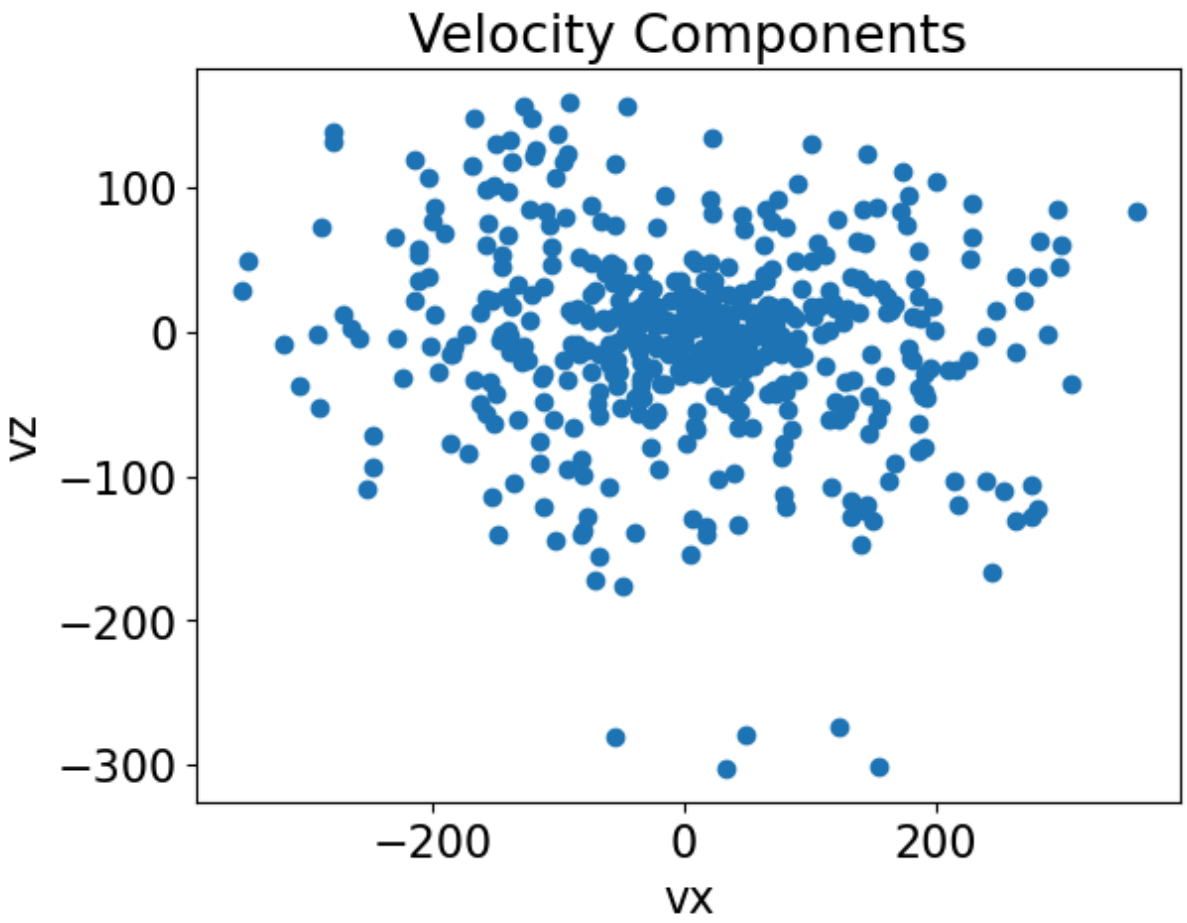
In [65]:
```
# check for missing values
print("Missing values :\n", data.isnull().sum())
```

```
Missing values :
 source_id_1    0
vx              0
vy              0
vz              0
En              0
Lz              0
apogee          0
feh             0
alpha           0
dtype: int64
```

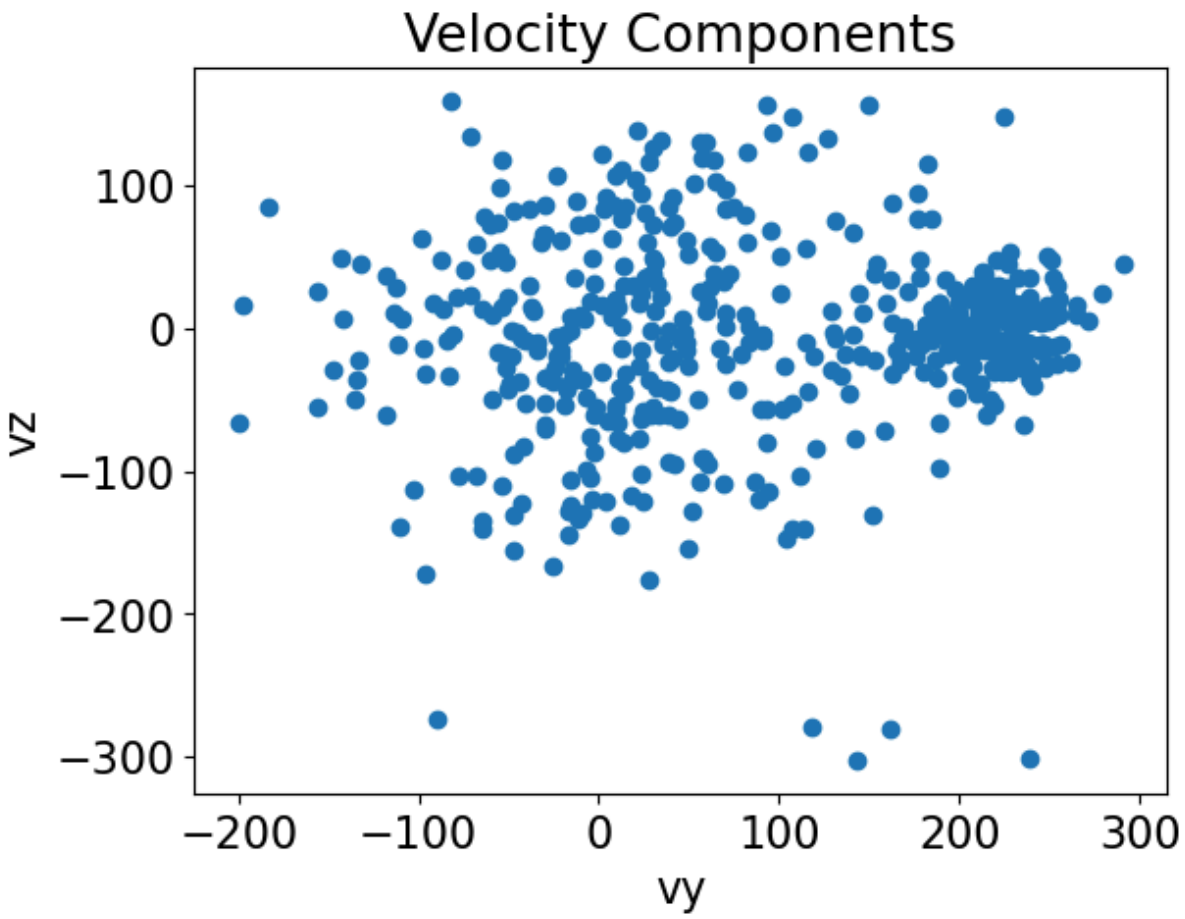
In [ ]:
```
sns.pairplot(data.drop(columns=['source_id_1']), diag_kind='kde', plot_kws=
{'alpha': 1})
plt.show()
```



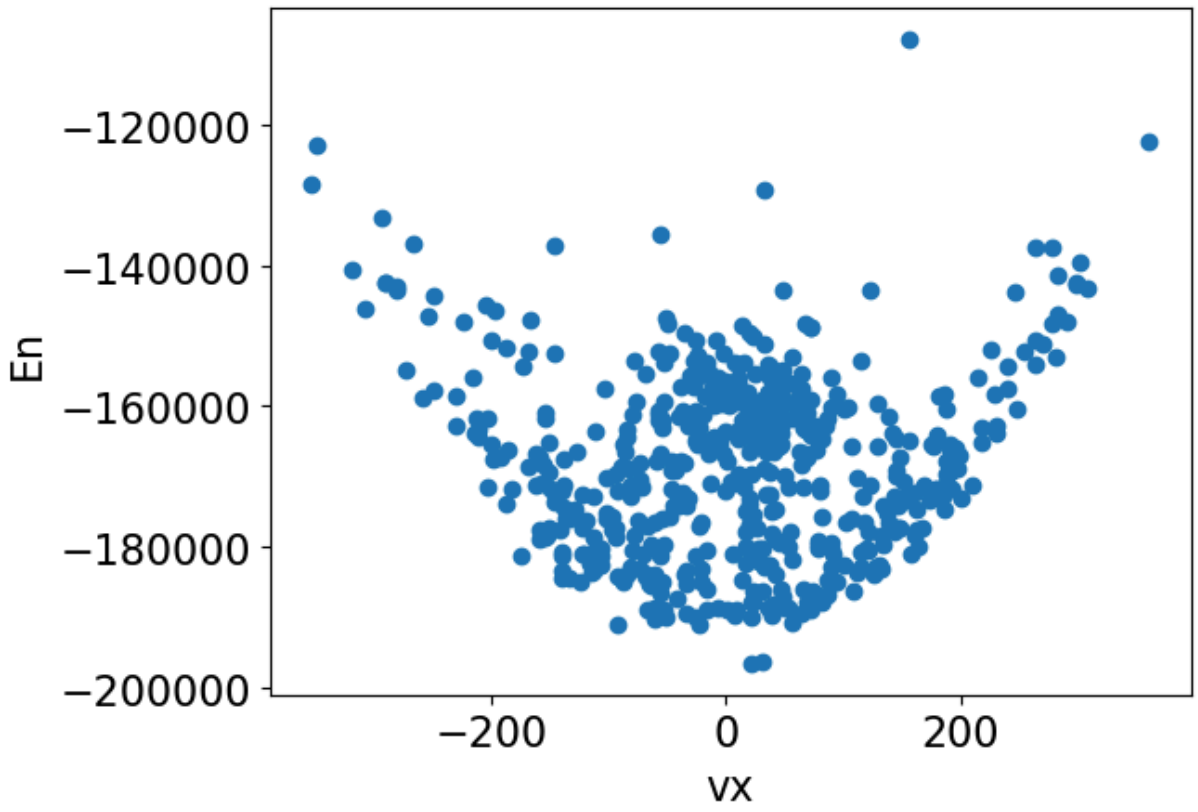
In [77]:
```
plt.scatter(data.vx,data.vz)
plt.xlabel('vx')
plt.ylabel('vz')
plt.title('Velocity Components')
plt.show()
```



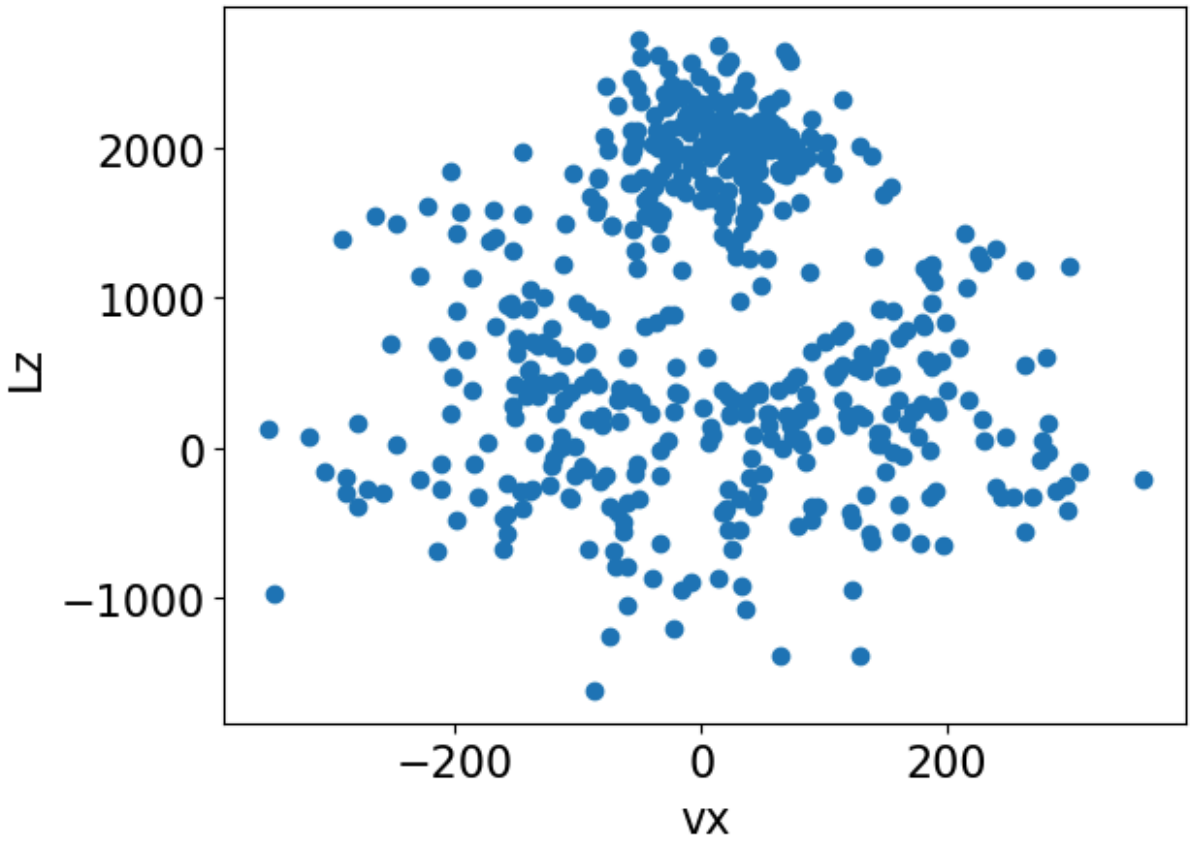
In [78]:
```
plt.scatter(data.vy,data.vz)
plt.xlabel('vy')
plt.ylabel('vz')
plt.title('Velocity Components')
plt.show()
```



In [80]:
```
plt.scatter(data.vx,data.En)
plt.xlabel('vx')
plt.ylabel('En')
plt.show()
```

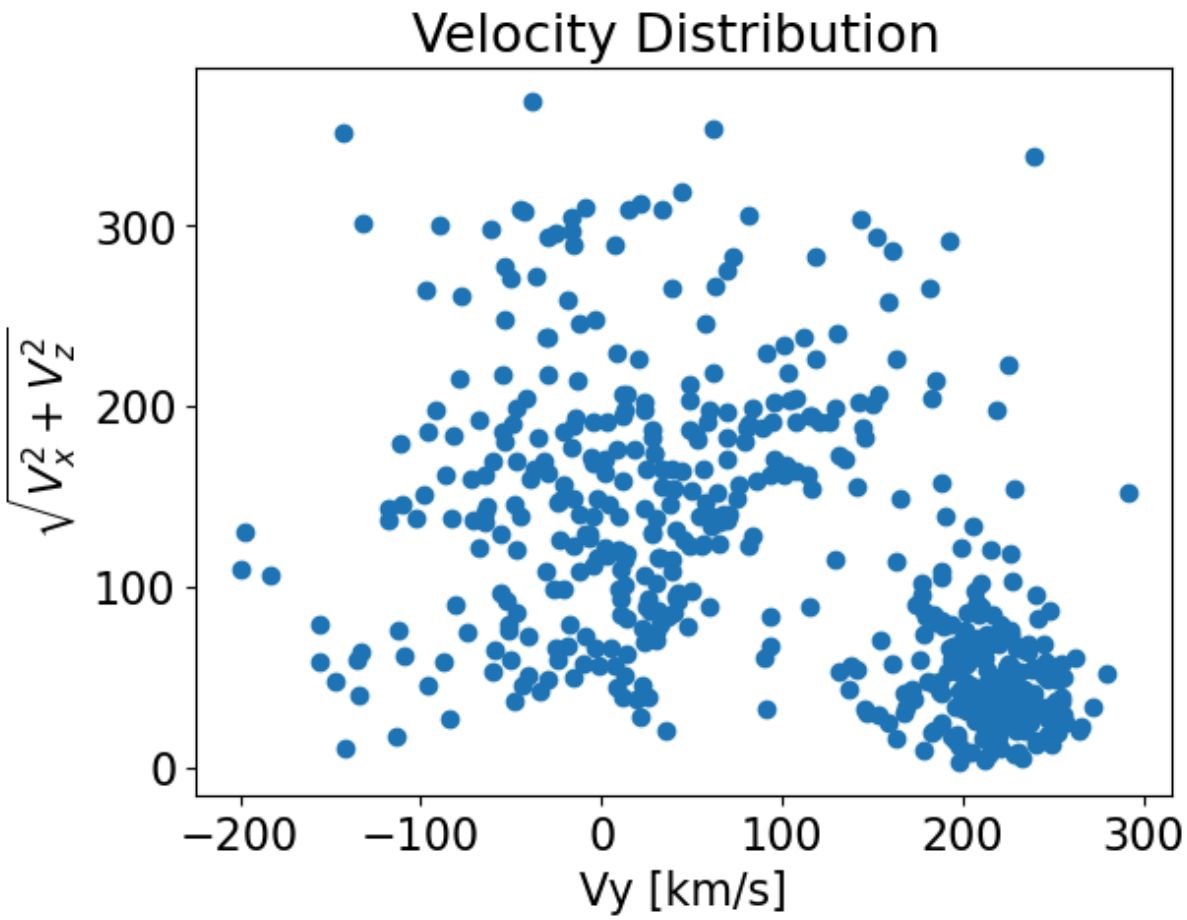


In [81]:
```
plt.scatter(data.vx,data.Lz)
plt.xlabel('vx')
plt.ylabel('Lz')
plt.show()
```

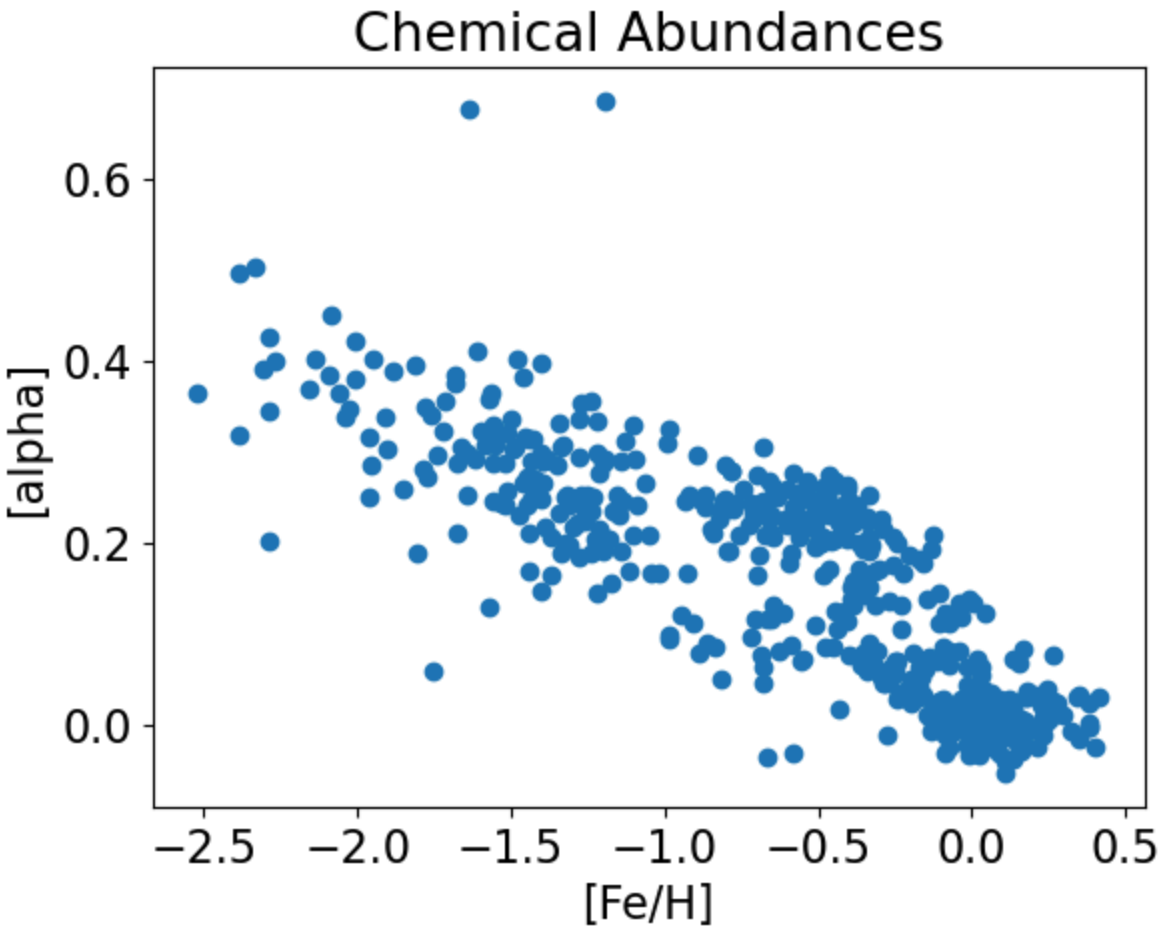


In [86]:
```
v_perp = np.sqrt(data['vx']**2 + data['vz']**2)

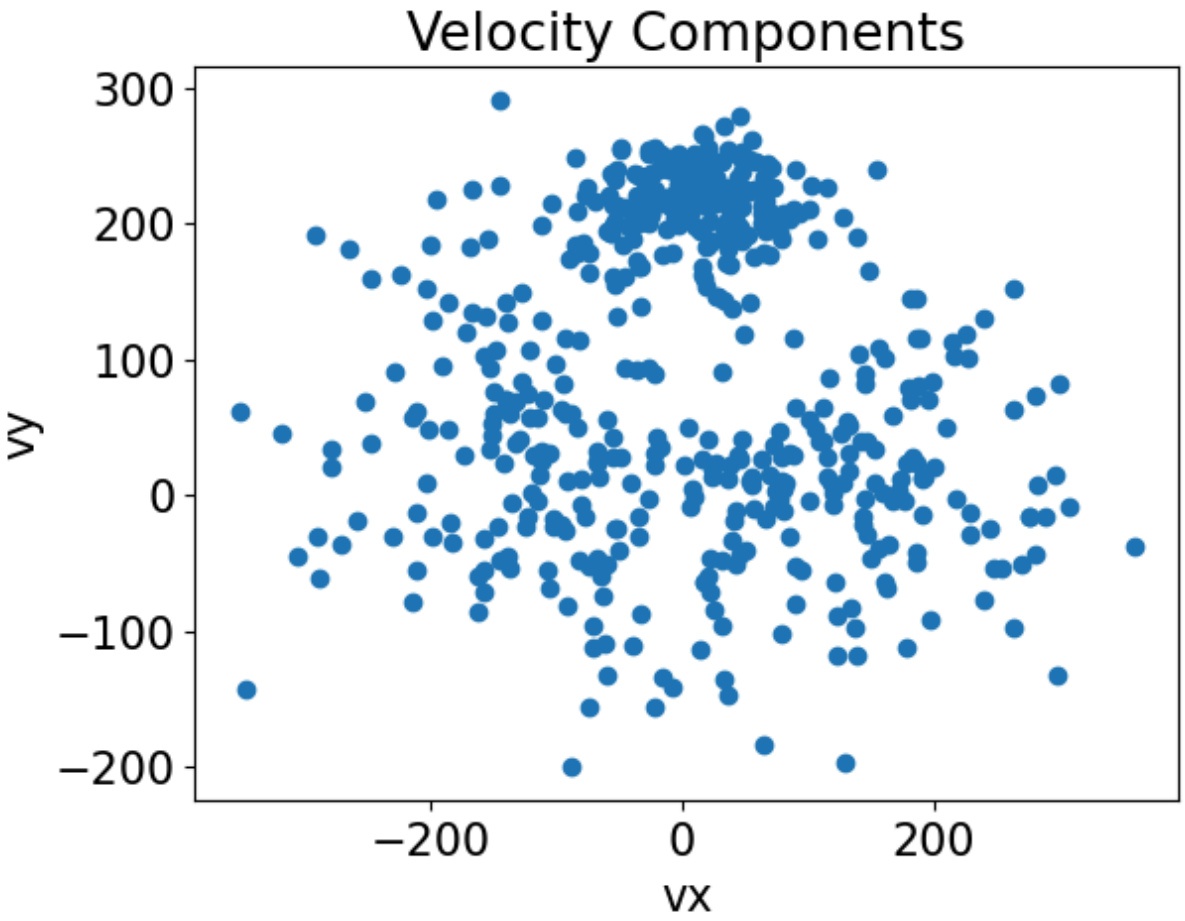
plt.scatter(data['vy'], v_perp)
plt.xlabel('Vy [km/s]')
plt.ylabel(r'$\sqrt{V_x^2 + V_z^2}$ ')
plt.title('Velocity Distribution ')
plt.show()
```



In [163…
```
plt.scatter(data['feh'], data['alpha'])
plt.xlabel('[Fe/H]')
plt.ylabel('[alpha]')
plt.title('Chemical Abundances')
plt.show()
```



In [74]:
```
plt.scatter(data['vx'], data['vy'])
plt.xlabel('vx')
plt.ylabel('vy')
plt.title('Velocity Components')
plt.show()
```



There is clear potential for clusterin in this dataset, sisible in two key projections (Also see the Helmi et al paper)

- Velocity space v_y versus $\sqrt{v_x^2 + v_z^2}$. There is a separation between cincentrated cluster of stars near low velocities and a more sipersed cluster near negative v_y values.
- Chemistry feh versus alpha. There are at least two distinct clusters !!

3.4 Prepare Data

The data is mostly prepared already.

3.5 Select model and train

Question 15

(4 pts)

Choose a clustering algorithm and try to find clusters in the data. You can use any clustering algorithm that you would like. Since this is unsupervised learning, we will not be splitting our data into training and testing.

In [132…
```
features=['vx','vy','vz','feh','alpha','Lz','En']
X_gaia=data[features]
```

In [133…
```
gmm_gaia=GaussianMixture(n_components=3,random_state=42,n_init=10)
gmm_gaia.fit(X_gaia)
```

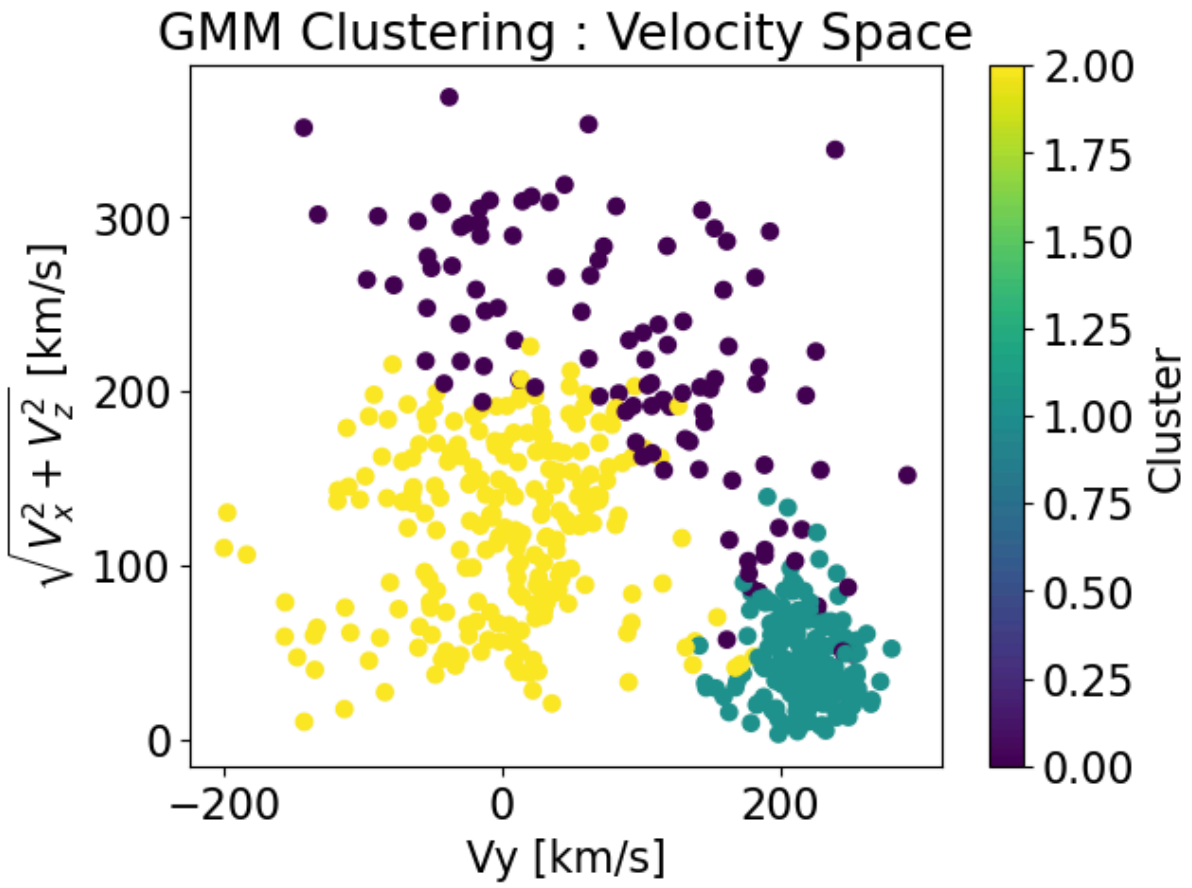
Out[133…
```
GaussianMixture(n_components=3, n_init=10, random_state=42)
```

In [134…
```
labels=gmm_gaia.predict(X_gaia)
print("Converged:",gmm_gaia.converged_)
print("Iterations:",gmm_gaia.n_iter_)
print("Cluster countts:",dict(zip(*np.unique(labels,return_counts=True))))
```

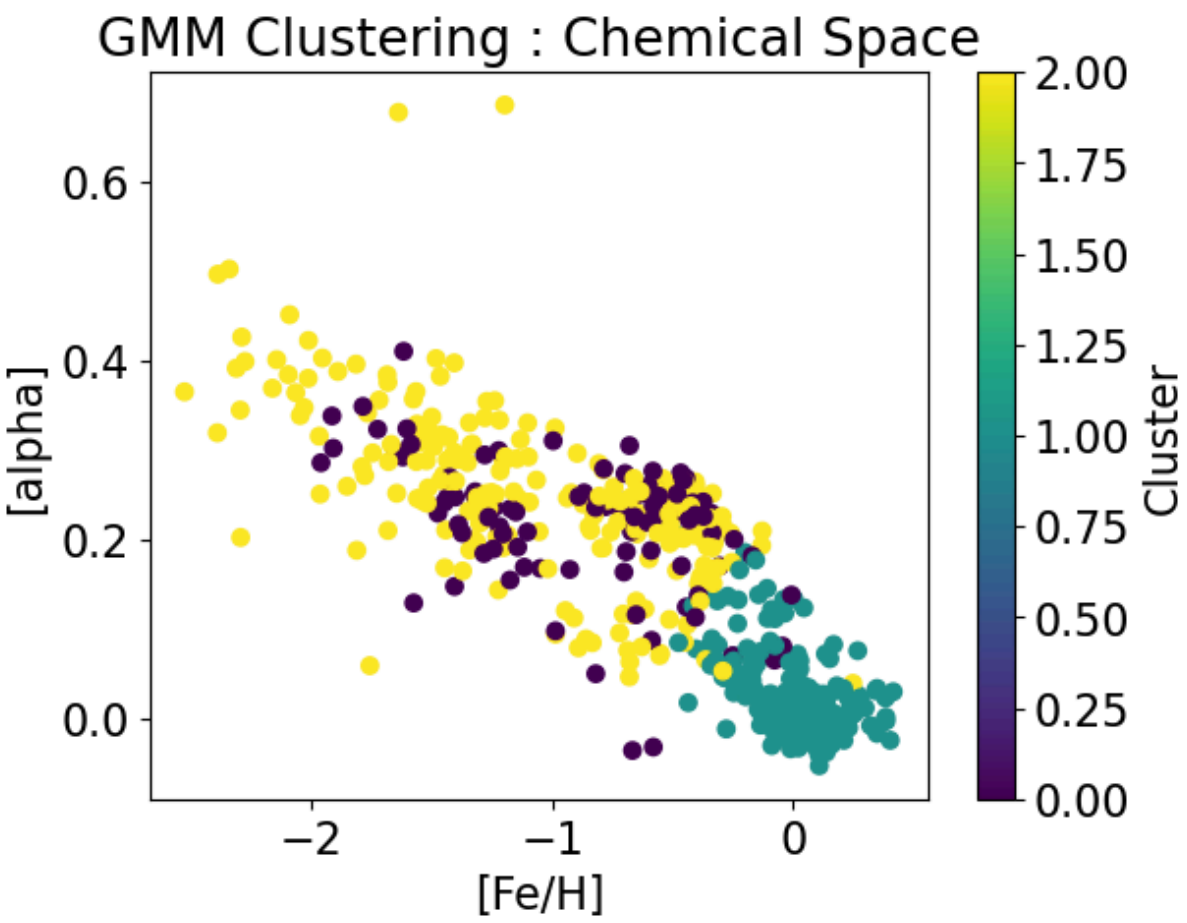
```
Converged: True
Iterations: 13
Cluster countts: {0: 106, 1: 193, 2: 227}
```

In [ ]:
```
v_perp = np.sqrt(X_gaia['vx']**2 + X_gaia['vz']**2)

plt.scatter(X_gaia['vy'], v_perp,c=labels,cmap='viridis')
plt.xlabel('Vy [km/s]')
plt.ylabel(r'$\sqrt{V_x^2 + V_z^2}$ [km/s]')
plt.title('GMM Clustering : Velocity Space')
plt.colorbar(label='Cluster')
plt.show()
```



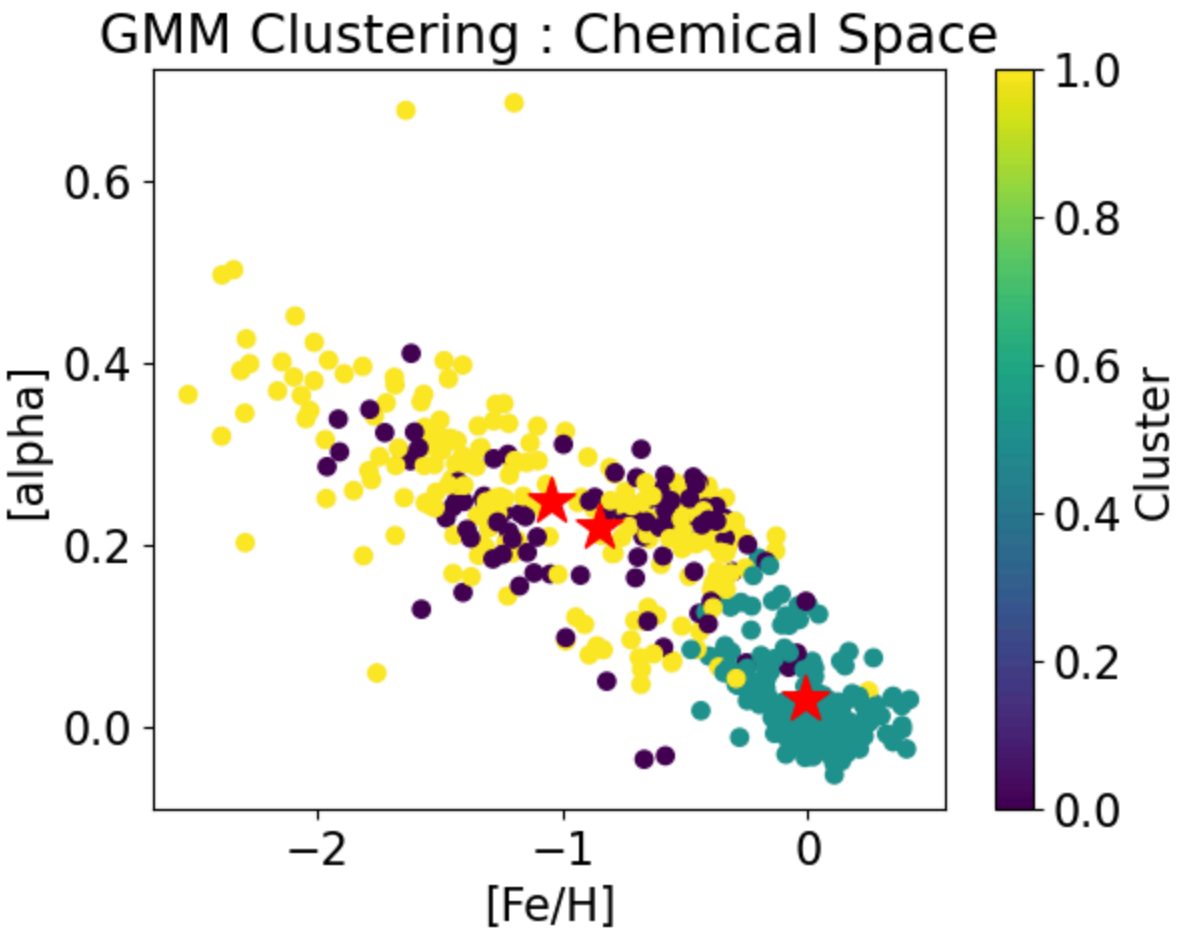
In [ ]:
```
plt.scatter(X_gaia['feh'], X_gaia['alpha'], c=labels, cmap='viridis')
plt.xlabel('[Fe/H]')
plt.ylabel('[alpha]')
plt.title('GMM Clustering : Chemical Space')
plt.colorbar(label='Cluster')
plt.show()
```



In [137…

```
gmmeans = gmm_gaia.means_

plt.scatter(X_gaia['feh'], X_gaia['alpha'], c=labels, cmap='viridis')
plt.scatter(gmmeans[:, 3], gmmeans[:, 4], color='red', marker='*', s=300)
plt.xlabel('[Fe/H]')
plt.ylabel('[alpha]')
plt.title('GMM Clustering : Chemical Space')
plt.colorbar(label='Cluster')
plt.show()
```

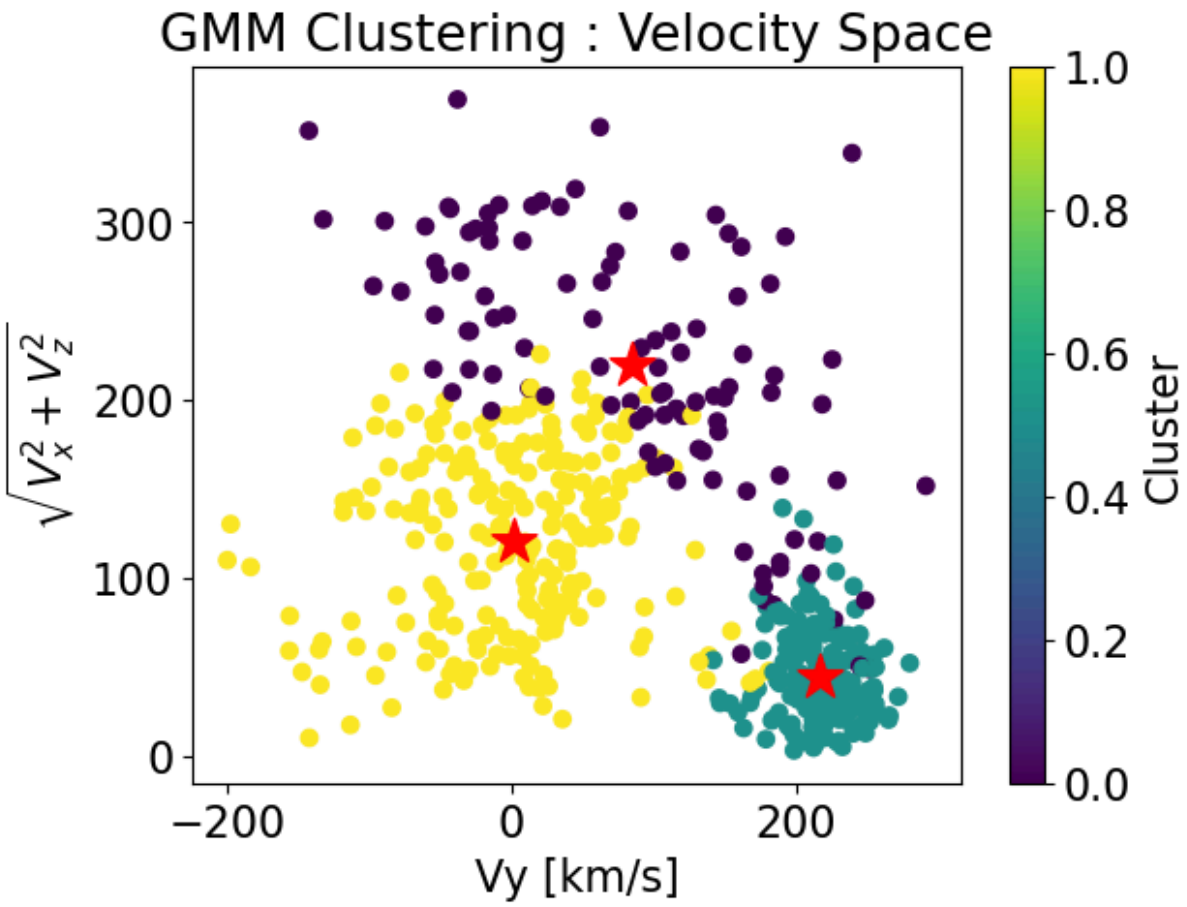


In [ ]:
```
v_perp = np.sqrt(X_gaia['vx']**2 + X_gaia['vz']**2)

plt.scatter(X_gaia['vy'], v_perp, c=labels, cmap='viridis')


for i in range(3):
    mask = labels == i
    plt.scatter(X_gaia['vy'][mask].mean(), v_perp[mask].mean(),
                color='red', marker='*', s=300)

plt.xlabel('Vy [km/s]')
plt.ylabel(r'$\sqrt{V_x^2 + V_z^2}$')
plt.title('GMM Clustering : Velocity Space')
plt.colorbar(label='Cluster')
plt.show()
```



Question 16

(4 pts)

Discuss the properties of the clusters that the algorithm picked out. Visualize the clusters using the different features

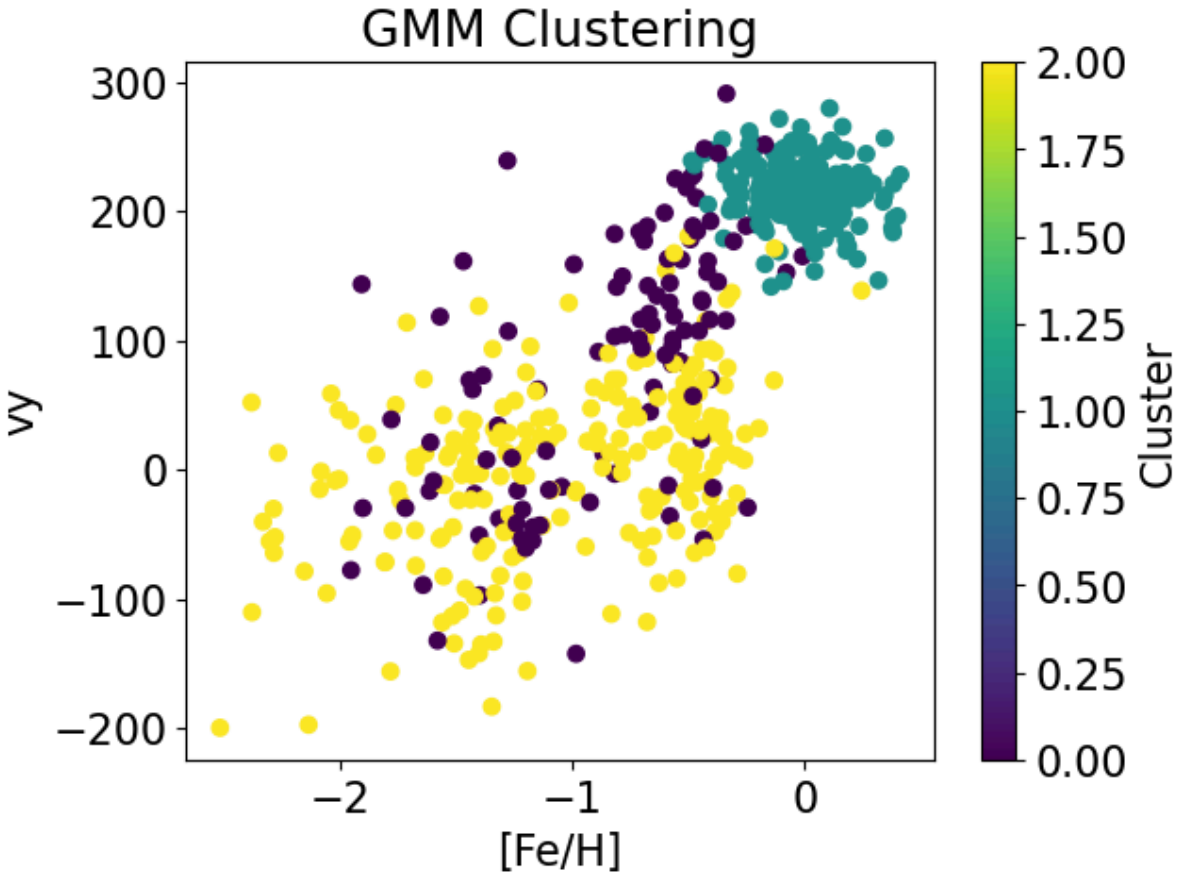
- Cluster 2 : This is the most distinct cluster !! Tightly grouped at high positive v_y and near zero $v_{\rm perp}$ in velocity space and at high feh, low alpha. The algorithms had no trouble separating this group - it is well separated from the others in both spaces
- Cluster 1 (yellow) - middle cluster. it is centered around $v_y \sim 0$ and moderate $v_{\rm perp}$ and sits at intermediate feh and alpha. This is the largest group !
- Cluster 0 (purple) - this is most dispersed nd dilute cluster. it is scattered at high $v_{\rm perp}$ and spread across all v_y values with low feh and high alpha. The cneter sits higher up in the velocity space.

Overall, the algorithm did very well at findign three clusters !!! The blobs are well separated (it depends on which features we use to visualize the scatter plots). The three

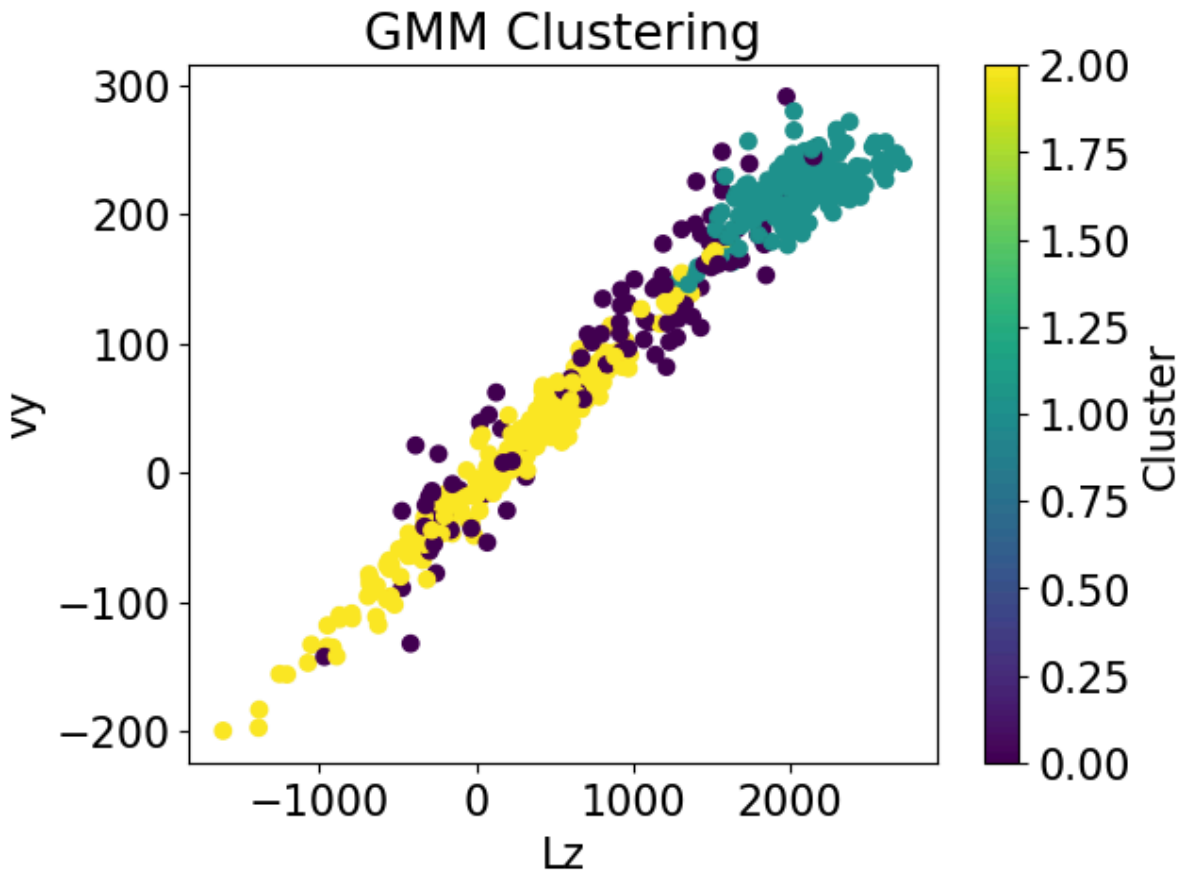
blobs are the cleareston the velocity space and not super clearly separated in chemical space !

Below we visualize more combinations of features and arrive at the same conlusion as outlined above !

In [141…
```
# visualize more combinations !!
plt.scatter(X_gaia['feh'], X_gaia['vy'], c=labels, cmap='viridis')
plt.xlabel('[Fe/H]')
plt.ylabel('vy')
plt.title('GMM Clustering')
plt.colorbar(label='Cluster')
plt.show()
```

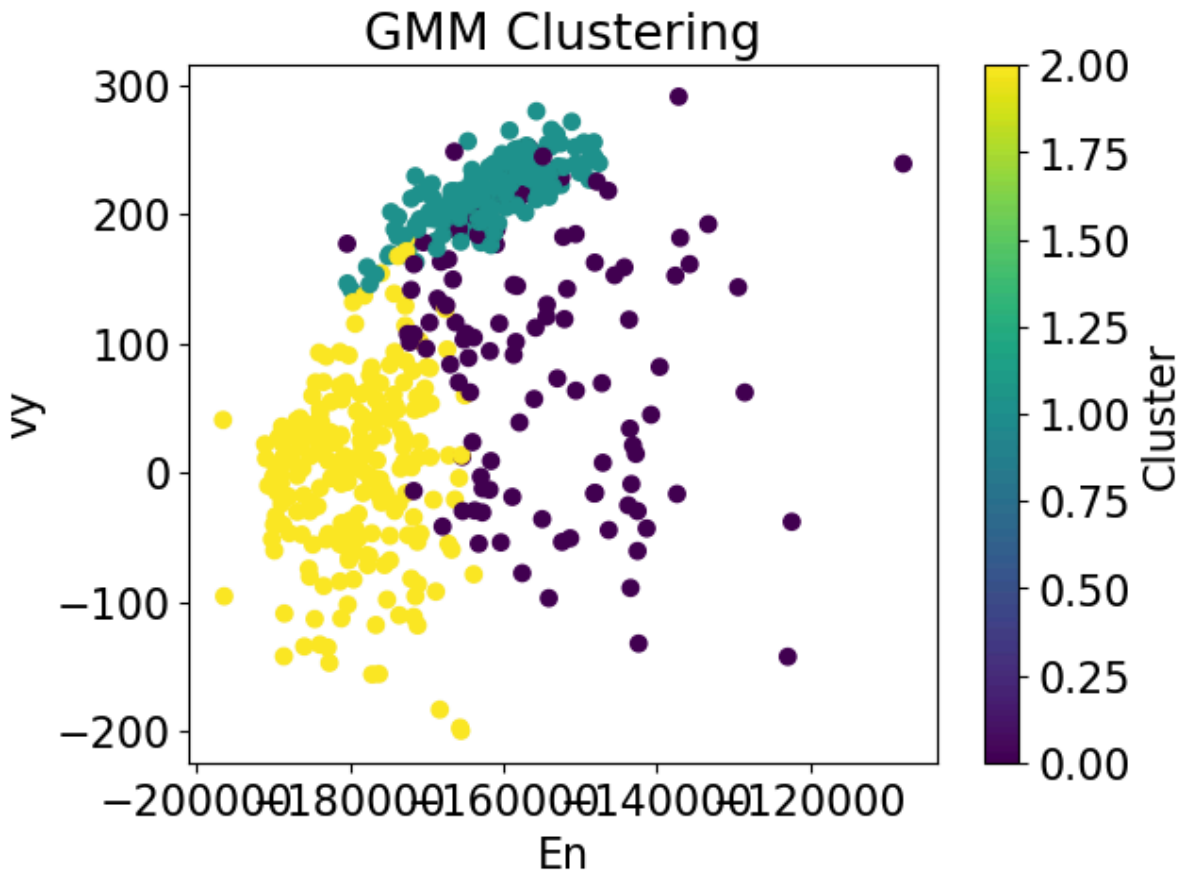


In [145…
```
# visualize more combinations !!
plt.scatter(X_gaia['Lz'], X_gaia['vy'], c=labels, cmap='viridis')
plt.xlabel('Lz')
plt.ylabel('vy')
plt.title('GMM Clustering')
plt.colorbar(label='Cluster')
plt.show()
```



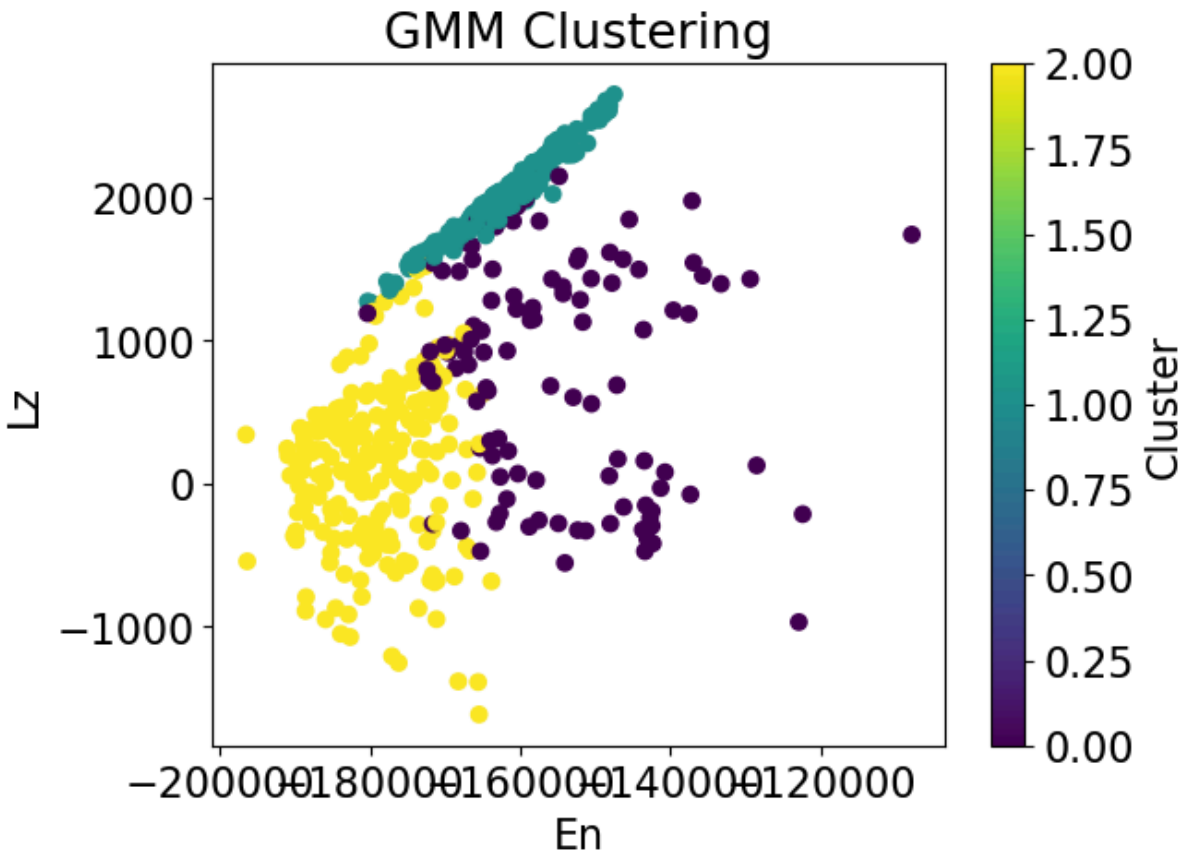
In [146…

```
# visualize more combinations !!
plt.scatter(X_gaia['En'], X_gaia['vy'], c=labels, cmap='viridis')
plt.xlabel('En')
plt.ylabel('vy')
plt.title('GMM Clustering')
plt.colorbar(label='Cluster')
plt.show()
```



In [147…

```
# visualize more combinations !!
plt.scatter(X_gaia['En'], X_gaia['Lz'], c=labels, cmap='viridis')
plt.xlabel('En')
plt.ylabel('Lz')
plt.title('GMM Clustering')
plt.colorbar(label='Cluster')
plt.show()
```

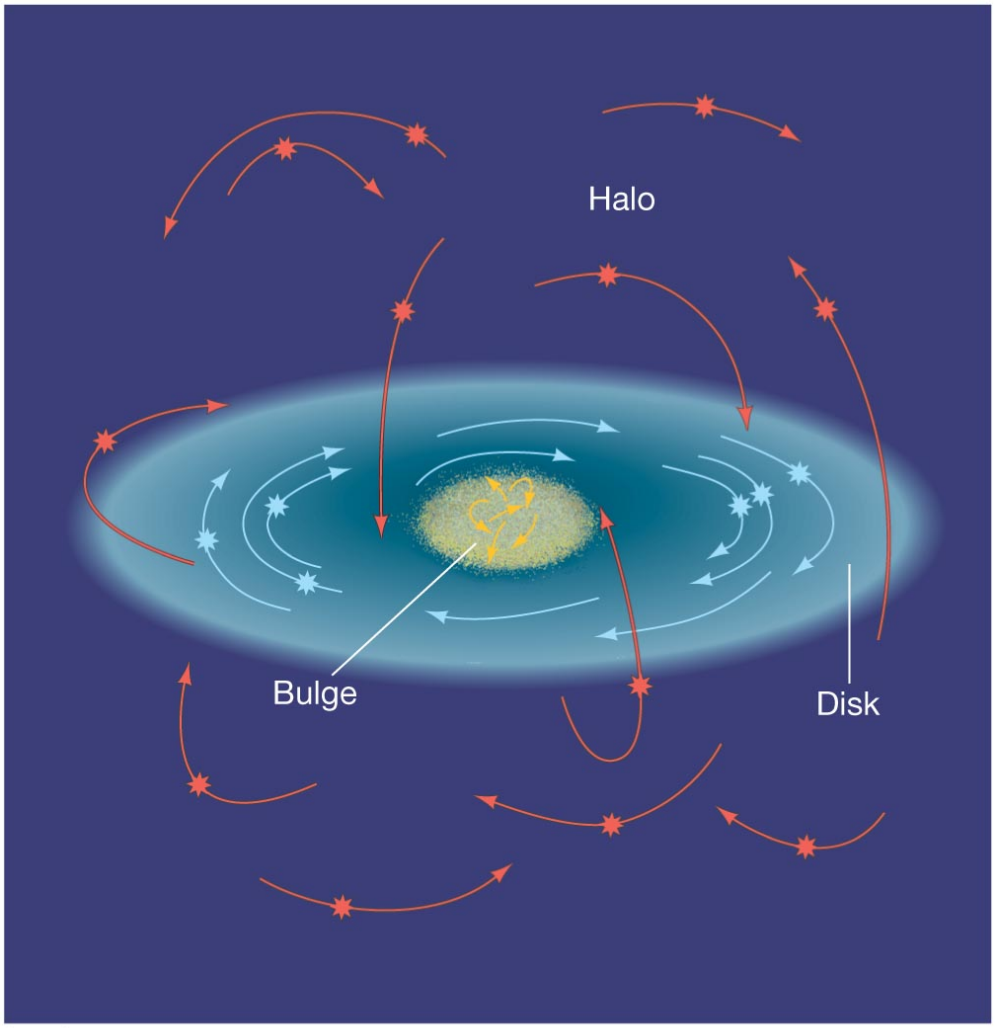


Question 17

(4 pts)

Test the clustering performance. Discuss what how you determined the optimal number of clusters

Hint: the Milky Way should have two components: stars in the disk of the Milky Way and stars in the halo. Stars in the Milky Way disk have iron abundance and alpha-elemental abundance close to that of the Sun. Stars in the Milky Way Halo tend to have low iron abundance ([Fe/H] < 0). The velocities of the disk and halo are also different. The image below shows some components of the Milky Way. The central region is called the bulge (there are no stars in this sample from that component).



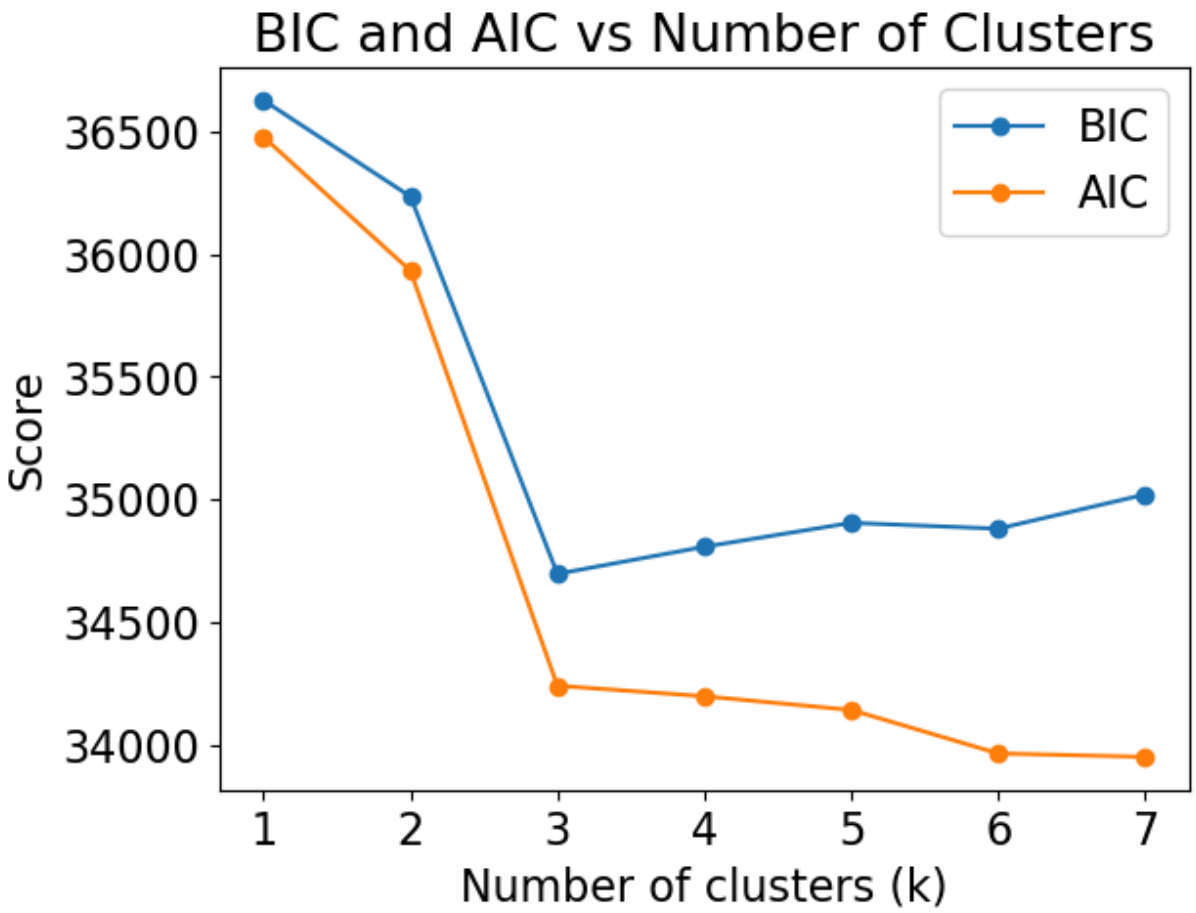
Copyright © 2008 Pearson Education, Inc., publishing as Pearson Addison-Wesley.

In [148…

```
# code here
bic_scores = []
aic_scores = []
k_range = range(1, 8)

for k in k_range:
    gmm = GaussianMixture(n_components=k, random_state=42, n_init=10)
    gmm.fit(X_gaia)
    bic_scores.append(gmm.bic(X_gaia))
    aic_scores.append(gmm.aic(X_gaia))

plt.plot(list(k_range), bic_scores, marker='o', label='BIC')
plt.plot(list(k_range), aic_scores, marker='o', label='AIC')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Score')
plt.title('BIC and AIC vs Number of Clusters')
plt.legend()
plt.show()
```



We use BIC and AIC to determine the optimal number of clusters. As in part 2, both score should show a minimum at 3 clusters, which aligns well with the physical expectation from the visualizations above !!! Three clusters is therefore both statistically justified by BIC/AIC and physically motivaed by the known componens of the Milky way ! We expect milky way to have two components (disk and halo) and on top oft that we are looking a third distinct population !

3.6 Fine tune model

Question 18

(5 pts)

Try another clustering algorithm and compare its performance with the previous method. Are there differences in how the clustering works with different number of features? What features are most helpful to distinguish clusters?

Hint: you can use the original paper Helmi et al. (2018): https://arxiv.org/abs/1806.06038 as a guide for how to think about the features. The original paper did not use clustering algorithms, but rather knowledge about the Milky Way's properties to find an extra component that did not belong.

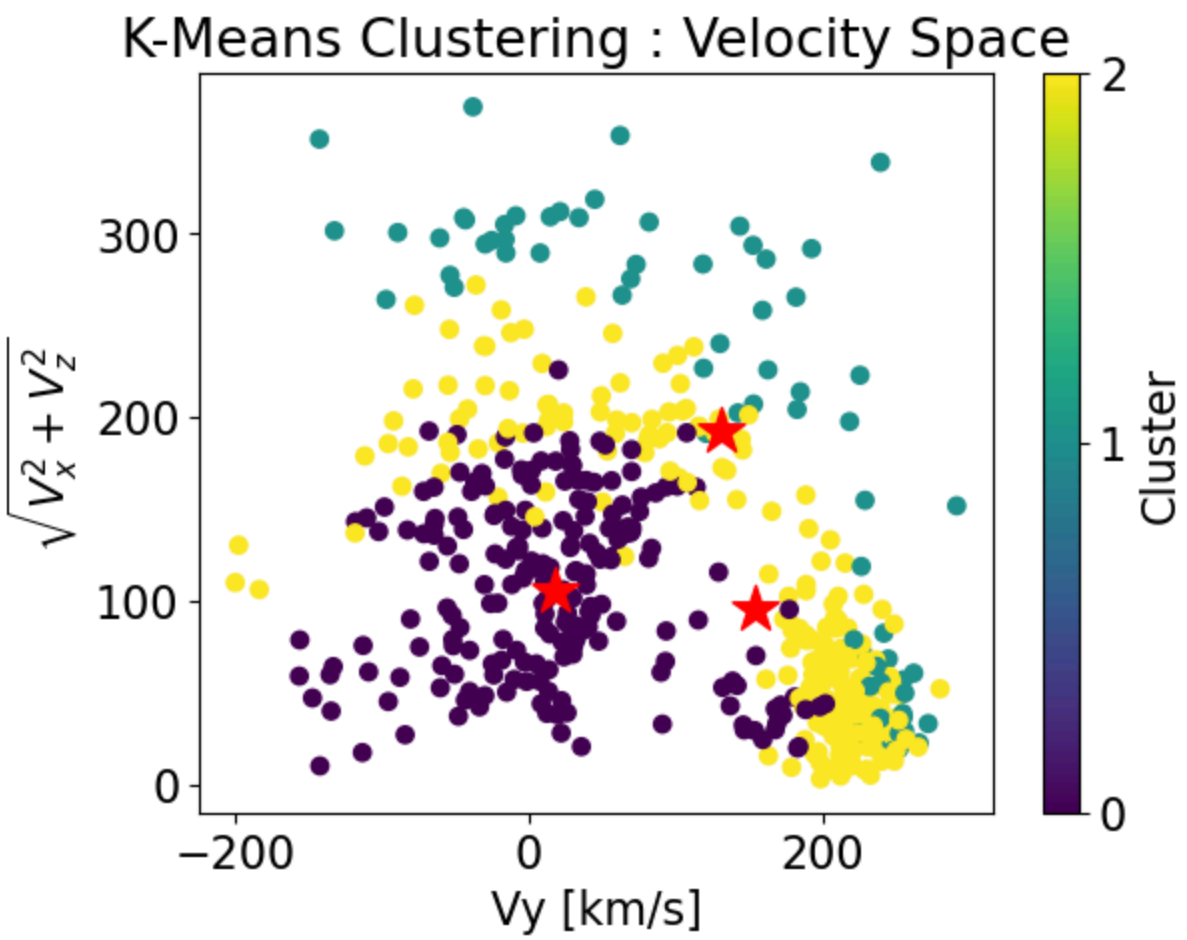
Use K-Means instead of Guassian Mixture

In [160…
```
# code here
kmeans_gaia = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_gaia.fit(X_gaia)
labels_km = kmeans_gaia.predict(X_gaia)
```

In [ ]:
```
fig, ax = plt.subplots()
scatter = ax.scatter(X_gaia['vy'], v_perp, c=labels_km, cmap='viridis',
vmin=0, vmax=2)
plt.colorbar(scatter, label='Cluster', ticks=[0, 1, 2])

for i in range(3):
    mask = labels_km == i
    ax.scatter(X_gaia['vy'][mask].mean(), v_perp[mask].mean(),
                color='red', marker='*', s=300)

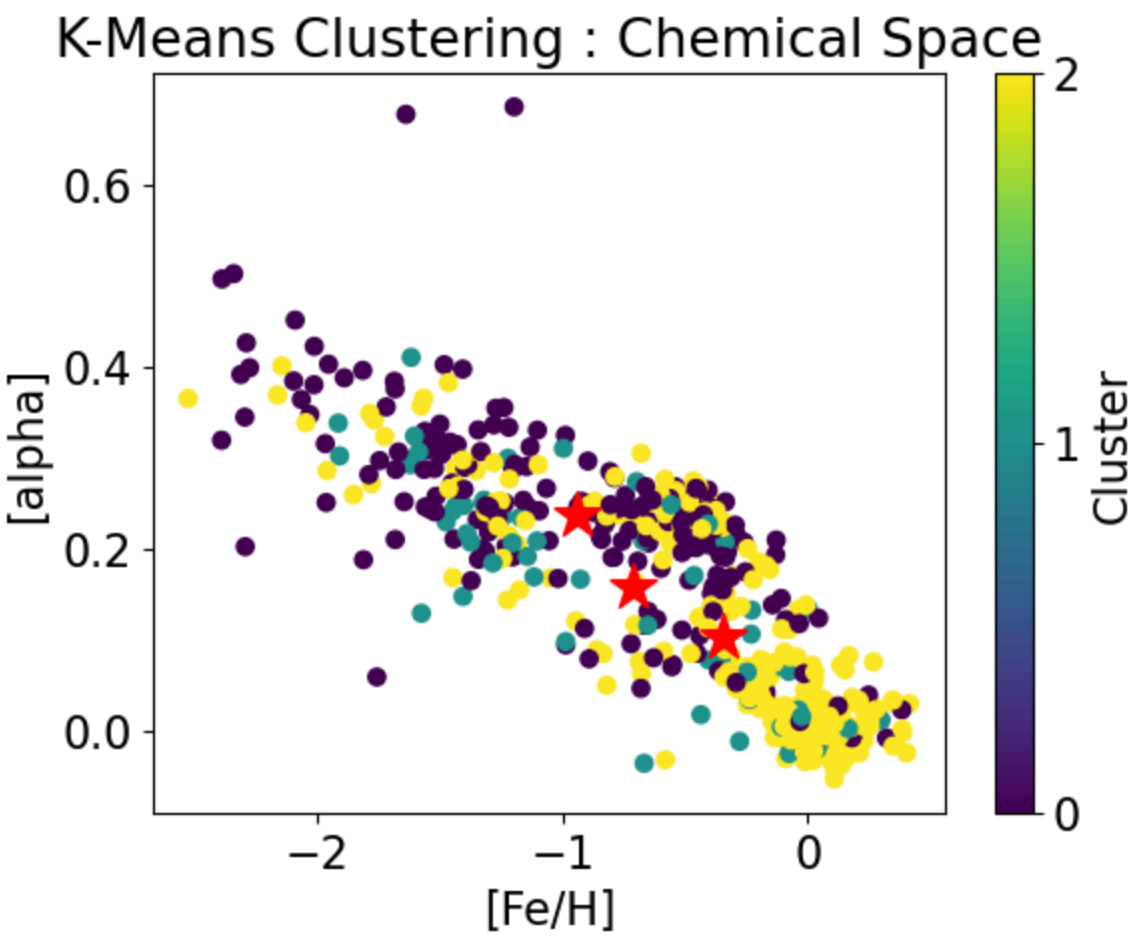
plt.xlabel('Vy [km/s]')
plt.ylabel(r'$\sqrt{V_x^2 + V_z^2}$')
plt.title('K-Means Clustering : Velocity Space')
plt.show()
```



In [ ]:
```
fig, ax = plt.subplots()
scatter = ax.scatter(X_gaia['feh'], X_gaia['alpha'], c=labels_km,
cmap='viridis', vmin=0, vmax=2)
plt.colorbar(scatter, label='Cluster', ticks=[0, 1, 2])

for i in range(3):
    mask = labels_km == i
    ax.scatter(X_gaia['feh'][mask].mean(), X_gaia['alpha'][mask].mean(),
                color='red', marker='*', s=300)

plt.xlabel('[Fe/H]')
plt.ylabel('[alpha]')
plt.title('K-Means Clustering : Chemical Space')
plt.show()
```



In [165…

```
inertia_scores = []
silhouette_scores = []
k_range = range(2, 10)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_gaia)
    inertia_scores.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_gaia, kmeans.labels_))

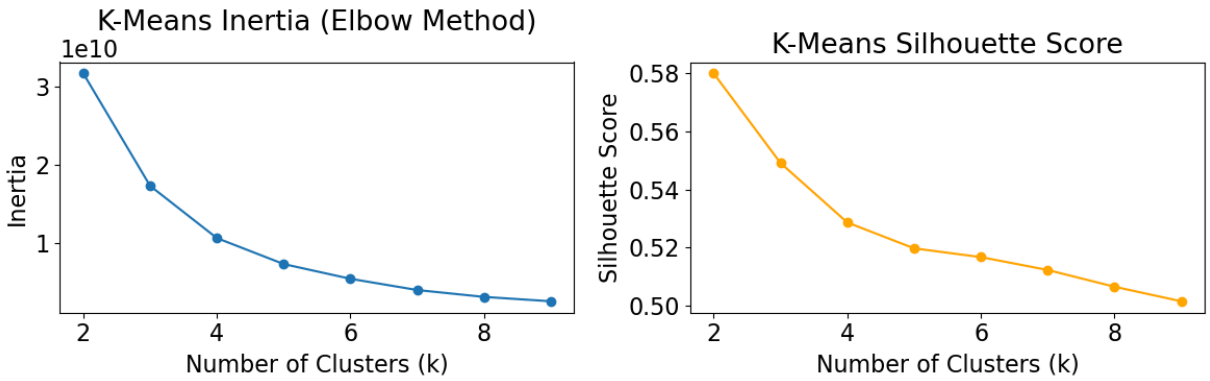
fig, axes = plt.subplots(1, 2, figsize=(12, 4))


axes[0].plot(list(k_range), inertia_scores, marker='o')
axes[0].set_xlabel('Number of Clusters (k)')
axes[0].set_ylabel('Inertia')
axes[0].set_title('K-Means Inertia (Elbow Method)')

axes[1].plot(list(k_range), silhouette_scores, marker='o', color='orange')
axes[1].set_xlabel('Number of Clusters (k)')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('K-Means Silhouette Score')

plt.tight_layout()
plt.show()
```

```
print(f"Optimal k by silhouette (highest score): k = {list(k_range)
[silhouette_scores.index(max(silhouette_scores))]}")
```




```
Optimal k by silhouette (highest score): k = 2
```

K-means is worse than Gaussian Mixture model !!! Apart from the fact that it gives us two cluster which contrasts our physical intuition and the clean three clusters demarcated by GMM, we do not see an "elbow" in the inertia and silhouette scores as we vary the number of clusters !! This means that the K-means algorithm is not comprehending the structure of the underlying data distribution !

Change the number of features considered !

Consider only features vy, feh and Lz

In [168…

```
less_features=['vy','feh','Lz']
less_X_gaia=data[less_features]
less_gmm_gaia=GaussianMixture(n_components=3,random_state=42,n_init=10)
less_gmm_gaia.fit(less_X_gaia)
less_labels=less_gmm_gaia.predict(less_X_gaia)
print("Converged:",less_gmm_gaia.converged_)
print("Iterations:",less_gmm_gaia.n_iter_)
print("Cluster
countts:",dict(zip(*np.unique(less_labels,return_counts=True))))
```

```
Converged: True
Iterations: 34
Cluster countts: {0: 165, 1: 200, 2: 161}
```

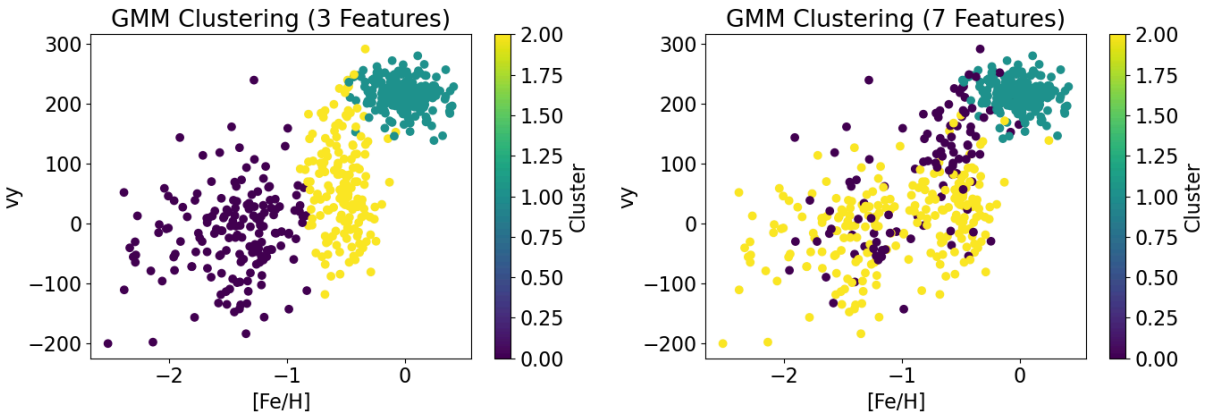
In [171…

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Reduced features (3 features)
axes[0].scatter(less_X_gaia['feh'], less_X_gaia['vy'], c=less_labels,
cmap='viridis')
axes[0].set_xlabel('[Fe/H]')
axes[0].set_ylabel('vy')
axes[0].set_title('GMM Clustering (3 Features)')
plt.colorbar(axes[0].collections[0], ax=axes[0], label='Cluster')
```

```
# All features (7 features)
scatter = axes[1].scatter(X_gaia['feh'], X_gaia['vy'], c=labels,
cmap='viridis')
axes[1].set_xlabel('[Fe/H]')
axes[1].set_ylabel('vy')
axes[1].set_title('GMM Clustering (7 Features)')
plt.colorbar(scatter, ax=axes[1], label='Cluster')

plt.tight_layout()
plt.show()
```



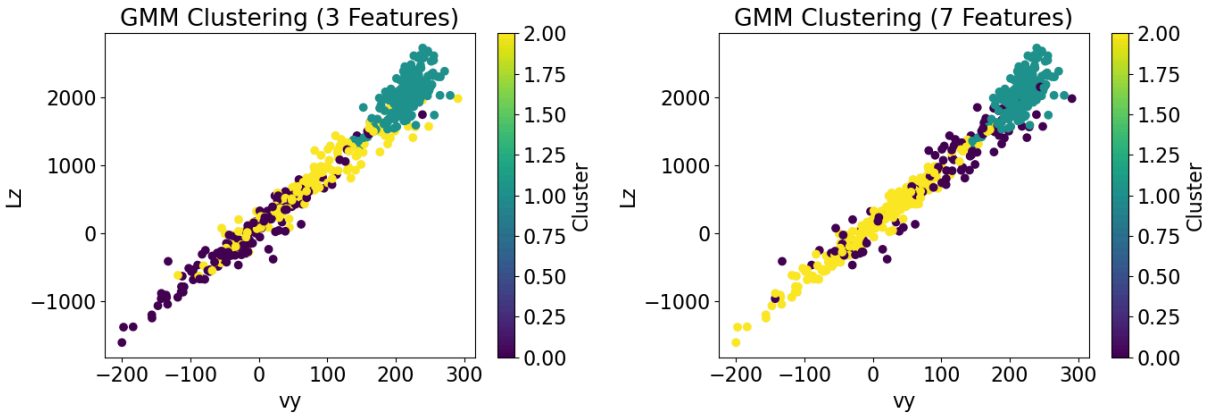
In [172…

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Reduced features (3 features)
axes[0].scatter(less_X_gaia['vy'], less_X_gaia['Lz'], c=less_labels,
cmap='viridis')
axes[0].set_xlabel('vy')
axes[0].set_ylabel('Lz')
axes[0].set_title('GMM Clustering (3 Features)')
plt.colorbar(axes[0].collections[0], ax=axes[0], label='Cluster')

# All features (7 features)
scatter = axes[1].scatter(X_gaia['vy'], X_gaia['Lz'], c=labels,
cmap='viridis')
axes[1].set_xlabel('vy')
axes[1].set_ylabel('Lz')
axes[1].set_title('GMM Clustering (7 Features)')
plt.colorbar(scatter, ax=axes[1], label='Cluster')

plt.tight_layout()
plt.show()
```



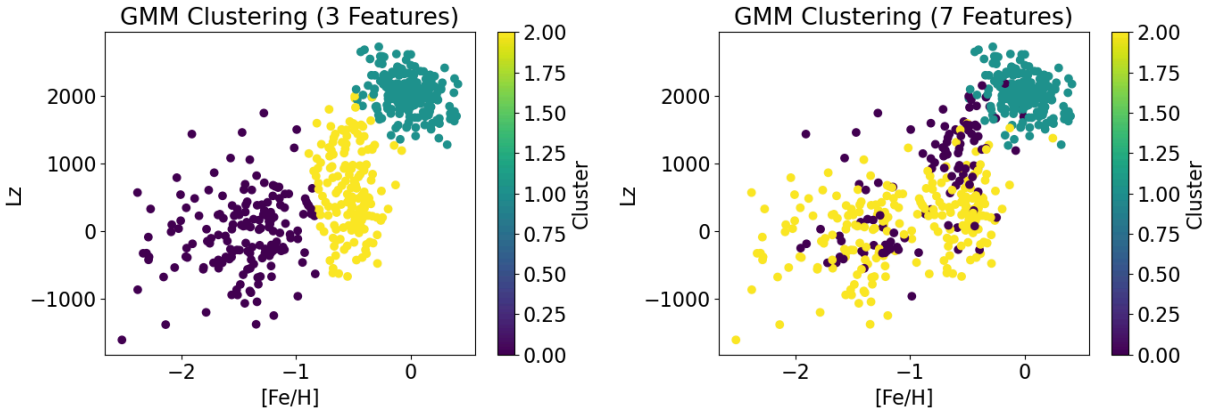
In [173…

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Reduced features (3 features)
axes[0].scatter(less_X_gaia['feh'], less_X_gaia['Lz'], c=less_labels,
cmap='viridis')
axes[0].set_xlabel('[Fe/H]')
axes[0].set_ylabel('Lz')
axes[0].set_title('GMM Clustering (3 Features)')
plt.colorbar(axes[0].collections[0], ax=axes[0], label='Cluster')

# All features (7 features)
scatter = axes[1].scatter(X_gaia['feh'], X_gaia['Lz'], c=labels,
cmap='viridis')
axes[1].set_xlabel('[Fe/H]')
axes[1].set_ylabel('Lz')
axes[1].set_title('GMM Clustering (7 Features)')
plt.colorbar(scatter, ax=axes[1], label='Cluster')

plt.tight_layout()
plt.show()
```



It is clear that if we are clever about the number and kind of features we choose for our clustering, the prediction quality increases substantially !!

In this exercise, we first naively considered all 7 features and obtained results which are not terrible. However in the second

part, based on the Helmi et al. paper, we chose our features judiciously and obtained the three clusters which are beautifully separated from each other !

3.7 Present solution

Question 19

(2 pts)

Discuss whether you can detect the presense of a dwarf galaxy in the data?

In [ ]:
```
# code here
```

- Cluster zero in purple las a low/negative Lz , very low feh. It has the signatures of a dwarf galaxy (Gaia Enceladus) !! This cluster had zero stars with near zero and negative Lz, meaning they orbit with no net rotation. Based on the paper, it is impossible for stars born in the Milky Way disk or halo. Thus these are starts from an accreted galaxy that merged sometime in the past !
- Low metallicity. Purple cluster zero stars reach feh ~ -2.5, which, based on the paper is consistent with a smaller dwarf galaxy that had less star formation and chemical enrichment !

Question 20

(2 pts)

How does your clustering solution compared with the original paper?

In the lab directory there's a file called `gaia_apogee_match_flag.csv` that is the same as the file you loaded, except there is a new column called `flag`. This flag is based on the identification of the stars from the Helmi et al. (2018) paper.

The flag values are: 0 = Milky Way disk stars 1 = Milky Way halo stars 2 = stars from the dwarf galaxy

Note that the identification from the Helmi et al. (2018) paper is based on some specific thresholds in energy and angular momentum, so a clustering algorithm may give more nuanced assignments to the component that each star belongs to.

In [ ]:
```
# code here
```

In [190…
```
# Check cluster centers to confirm identity
for i in range(3):
    mask = less_labels == i
    print(f"Cluster {i}: mean feh={less_X_gaia['feh'][mask].mean():.2f},
mean Lz={less_X_gaia['Lz'][mask].mean():.0f}, count={mask.sum()}")
```

```
Cluster 0: mean feh=-1.46, mean Lz=-31, count=165
Cluster 1: mean feh=-0.02, mean Lz=2024, count=200
Cluster 2: mean feh=-0.53, mean Lz=621, count=161
```

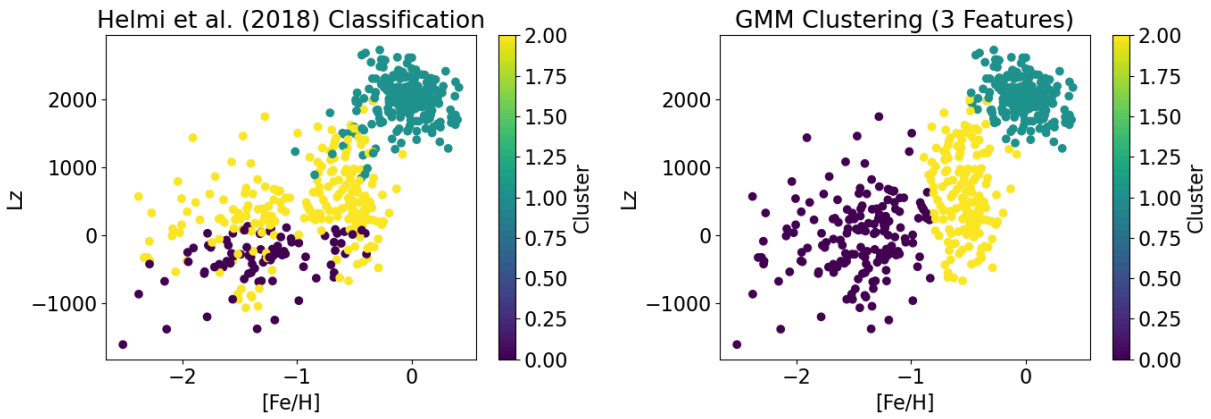
In [191…
```
# Remap Helmi flags to match our GMM cluster numbering
# Helmi: 0=disk, 1=halo, 2=dwarf ; My GMM: 1=disk, 2=halo, 0=dwarf
flag_remap = {0: 1, 1: 2, 2: 0}
data_flag['flag_remapped'] = data_flag['flag'].map(flag_remap)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Original paper flags (remapped)
axes[0].scatter(data_flag['feh'], data_flag['Lz'],
c=data_flag['flag_remapped'], cmap='viridis')
axes[0].set_xlabel('[Fe/H]')
axes[0].set_ylabel('Lz')
axes[0].set_title('Helmi et al. (2018) Classification')
plt.colorbar(axes[0].collections[0], ax=axes[0], label='Cluster')

# My GMM (with 3 features ) clustering
scatter = axes[1].scatter(less_X_gaia['feh'], less_X_gaia['Lz'],
c=less_labels, cmap='viridis')
axes[1].set_xlabel('[Fe/H]')
axes[1].set_ylabel('Lz')
axes[1].set_title('GMM Clustering (3 Features)')
plt.colorbar(scatter, ax=axes[1], label='Cluster')

plt.tight_layout()
plt.show()
```



- Disk (Cluster 1) : Near perfect agreement. The tight concentrated group at high Lz and high feh is identified by both methods ! This is the most well -speparated cluster and both approaches have no trouble finding it.

- Dwarf galaxy (cluster 0) : Good agreement on the core population at low feh and Lz close to zero.
- Halo (Yellow cluster 1) : I think the biggest difference is here ! In the Helmi et al paper, these ar more spread out and mixed with the dqarf galaxy. In my GMM model, the halo and the dwarf galaxy are more cleanly separated. GMM assigns fewwer stars to the dqarf galaxy in that intermediate region .

Both methods successfully detect Gaia-Enceladus. I think the key difference is that the paper's hard thresholds contrast with GMM's probabilistic boundaries resulting in a difference between the two methods !

Lab Survey and Submit

Please take this lab survey to give us feedback to improve the lab! Survey Link: https://forms.gle/28yywKq2tYV15YXE9

Remeber to restart your kernel and run all cells before submitting!